Trinity College Dublin
Coláiste na Tríonóide, Baile Átha Cliath
The University of Dublin

School of Computer Science and Statistics

RGB hand pose estimation by learning to mimic hands with a parametric hand model

Samuel McCann

A Dissertation submitted in partial fulfilment
of the requirements for the degree of
MCS (Computer Science)

Supervisor: Gerard Lacey

Submitted to the University of Dublin, Trinity College, April, 2020

Declaration

I hereby declare that this project is entirely my own work and that it has not been submitted as an exercise for a degree at this or any other university.

Signed: _____ Date: _____

Summary

3D Hand Pose estimation enables new forms of intuitive human-computer interaction. This thesis evaluates a model-based deep learning approach to hand pose estimation. We teach a deep network to control the MANO model, a parametric kinematically correct hand-model. We think a deep network will be capable of recognizing and mimicking hand poses in RGB images with the MANO model for reliable 3D RGB hand pose estimation.

Other methods break the task of hand pose estimation into multiple networks. We use a single network which is more efficient for real-time inference and simpler to train. We want to find out if this single-network MANO approach is comparative in accuracy to other approaches. We hypothesize the MANO approach is capable of tracking hands that are interacting with objects. Some methods fail to produce reasonable pose estimations when hands are partially hidden from the camera.

We evaluate the accuracy of our approach on the Stereo Tracking Benchmark. Our approach measured 12.5mm mean joint error on this benchmark, which places it in the middle of the pack relative to other RGB based methods on this benchmark. Using a single network enables real-time performance on weaker hardware compared to other multiple-network approaches, processing 55fps vs 16fps and 4fps for two other approaches.

We use the Obman dataset, a synthetic dataset made up of hands grasping objects, to evaluate the performance of this MANO approach with occluded hands. This approach was capable of tracking partially obscured hands. When compared to a dedicated hand-object pose estimation method, this approach falls behind in joint accuracy again, measuring 14.2mm of error vs a more accurate 11.6mm of mean joint error.

We present a model-based approach to RGB pose estimation using the MANO model. A single deep network, ResNet, can learn a mapping from RGB images to pose parameters for the MANO model. We liken the approach to a jack of all trades in hand pose estimation. When compared to the respective state of the art methods for hands and hand-object poses, the predictions are less accurate. This approach, however, can be trained on any public dataset, is more efficient for real-time pose estimation and does not fail on partially obscured hands.

RGB hand pose estimation by learning to mimic hands with a parametric hand model

Samuel McCann, MCS (Computer Science)
University of Dublin, Trinity College, April 2020

Supervisor: Gerard Lacey

Abstract

Using the MANO model, we investigate a model-based approach to RGB 3D hand pose estimation. The MANO model is a kinematically correct parametric hand model. We teach a deep network to control this hand model, mimicking hands in RGB images to infer hand pose.

We use a single deep network, ResNet, which is simpler to train and more efficient for real-time pose estimation compared to other multiple-network based approaches. We think using the MANO hand model will enable the network to infer hand poses from partially obscured hands interacting with objects, which some other methods fail to do. We evaluate the pose estimation accuracy of this approach relative to the state of the art and measure the ability for the network to infer hand poses on partially obscured hands.

In this thesis, we demonstrate a MANO model-based approach is capable of inferring 3D hand pose from RGB images, even for hands obscured from the camera when interacting with objects. We find this approach to be slightly less accurate than other approaches. It is, however, capable of tracking partially obscured hands and is more efficient for real-time pose estimation relative to other RGB based approaches.

Acknowledgements

First and foremost, I'd like to thank my supervisor, Dr Gerard Lacey, for providing me with the guidance and structure to produce this work.

I'd like to recognize the assistance of Tejo Chalasani and Daniel Dennis in getting started in this research area. I would also like to acknowledge my second reader, Dr Brendan Tangney, for his valuable feedback on my research. A thank you goes out to all of the lecturers and faculty in Trinity College Dublin responsible for the five years I've had studying here.

I'm sincerely grateful for my family and friends, without whom I would not have accomplished this. Thank you to my Dad, Alan, for inspiring and motivating me to reach this point in my life. To my Mum, Doris, for all her love and support throughout these years. A shout-out to my siblings for all the joy they add to life.

Contents

List of Figures

List of Tables

Nomenclature

$\vec{\beta}$	Shape vector parameters for MANO model
$\vec{\theta}$	Pose vector parameters for MANO model
AUC	Area Under Curve
CNN	Convolutional Neural Network
DoF	Degrees of Freedom
EPE	End-to-end Point Error
GT	Ground Truth
H	Hand image
HO	Hand-Object image
LBS	Linear Blend Skinning
MANO	hand **M**odel with **A**rticulated and **N**on-rigid def**O**rmations
MSE	Mean Squared Error
PCA	Principal Component Analysis
PCK	Percentage of Correct Key-points
PSO	Particle Swarm Optimization
RF	Random Forest
RGB	Red Green Blue
RoI	Region of Interest
ResNet	Residual Network
SMPL	Skinned Multi-Person Linear Model
SOTA	State Of The Art
STB	Stereo Tracking Benchmark dataset

1 Introduction

1.1 Motivation

Hands are one of our primary tools for interacting with the world around us. Hand pose estimation is already used in virtual reality (VR) and augmented reality (AR) to enable intuitive interactions with the surrounding virtual environments.

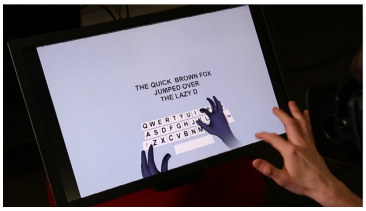
Other fields of intuitive human-computer interaction include natural language processing and body pose estimation. Natural language processing is already used commercially in devices like Amazon's Alexa, Google Home etc. Full body gestures have been used to control video games with body pose estimation since 2011 using the Kinect sensor.

Valve's new index controller gives the user the ability to interact with their hands in a VR game world by tracking their fingers, hand positions and global rotations, figure 1.1a. Leap motion has a hand tracking system for VR but it is limited to short-range tracking and requires a 3rd party depth sensor.

(a) Valve Index Controller

(b) Microsoft Hololens

(c) Sign language recognition



(d) VR Typing [6]

(e) VR piano [6]

(f) VR world interaction [6]

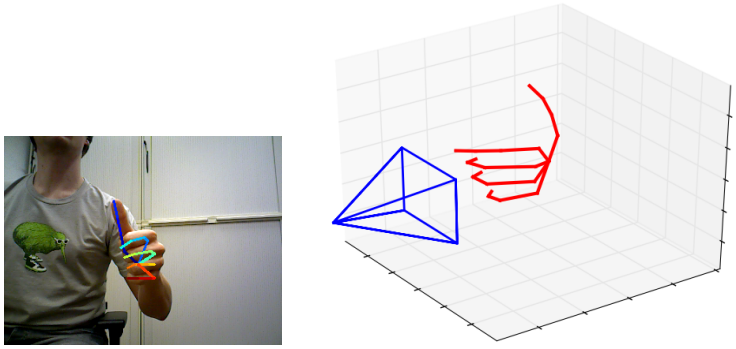
Figure 1.1: Examples of human-computer hand interactions

Hand tracking is still not preferred over hand-held controllers for VR/AR. We see the

specialized hardware and lack of reliability as the prohibiting factors. We want to investigate what could be done for a more reliable RGB based hand tracking method to make the examples featured in figure 1.1 a reality.

1.2 3D Hand Pose Estimation

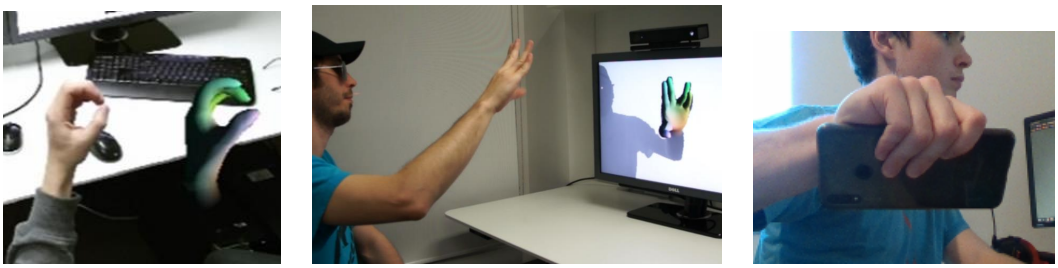
Making hand tracking convenient to use requires a lot of flexibility in the setup. Hand-held controllers, room tracking installations and multi-camera setups are limited in their applications and offer a high barrier for entry. Single depth sensors have been reasonably successful in hand tracking thus far. We think RGB hand tracking can provide an even lower barrier for entry with more versatility in how the hand tracking is used, which is why we focus on single-camera RGB hand tracking.



(a) 2D pose estimation (b) 3D pose estimation

Figure 1.2: 2D and 3D pose estimation example, from [1]

A single-camera gives a 2D image, from which we need to produce a 3D prediction. For simpler objects, 3D predictions are no longer a huge challenge, with AR phone applications like Pokemon Go already released to the public. 3D hands are more complicated to track. Different camera angles change the look of a hand pose dramatically. Many features on the hand are self-similar. Fingers, for example, can be quite challenging to distinguish.



(a) 1st Person with self-occlusion, from [8]

(b) 3rd Person Perspective, from [8]

(c) Object Occlusion

Figure 1.3: Various viewpoints and occlusions in hand pose estimation

Viewing hands from a single-camera viewpoint results in self and/or object occlusions. We consider a hand self-occluded when parts of the hand occlude other parts of the hand. Fingers can hide each other from the camera, example figure 1.3a. Object occlusions are caused by hands interacting with objects, like a mobile phone, figure 1.3c. Inferring joints and invisible features of the hand adds an additional layer of difficulty.

All these variables result in a high dimensional problem space. A robust solution is one that can produce good predictions under all kinds of viewpoints, articulations and partially obscured hands. It would be resistant to failure, producing a reasonable estimate rather than outright failing under uncertainty. A robust solution to all these challenges in hand pose estimation is an on-going research problem.

1.3 Research Objectives

In this thesis, we evaluate a hand-model based machine learning approach to RGB hand pose estimation. We will be teaching a deep network how to control a parametric hand model, the MANO model, figure 1.4, to mimic hands in RGB images.

Figure 1.4: The parametric MANO hand model from [2]

The MANO hand-model can be controlled by a vector of pose parameters to produce any kinematically correct hand articulation.

Methods that do not use a model-based approach to hand pose estimation rely on constraints and training to teach deep networks the kinematics of a hand. A kinematic understanding of the hand is particularly important for being able to infer hand pose under object occlusion, where important features of the hand may be hidden from view. We think this model-based approach in RGB imaging can be as accurate as other methods and be more reliable when hands are obscured from the camera.

Model-based approaches are successful with depth images [6, 8, 22]. We are going to evaluate a model-based approach in RGB.

We use ResNet for our deep network in this research. It was the first deep network to

surpass humans in the Imagenet classification challenge [23] and it has been validated in other hand pose estimation research. [7, 12, 20].

Our research objective is to evaluate this RGB MANO model-based deep learning approach. We train and test the model for overall prediction accuracy for isolated and occluded hands. We measure performance on the Stereo Tracking Benchmark (STB) [15] to compare overall accuracy to the state of the art. We use the synthetic hand and object Obman [12] dataset to examine how well the method handles occluded hands.

1.4 Overview

Chapter 2 is a review of current literature in 3D hand pose estimation. Both RGB and depth-based methods are reviewed. Public datasets and benchmarks are examined to properly evaluate this thesis.

Chapter 3 discusses the hypotheses and requirements driving the design of the implementation of the MANO based approach in this thesis.

Chapter 4 presents the implementation details of the MANO model-based approach for this research. Evaluations, training procedures and caveats to the implementation are detailed.

Chapter 5 reports the results from our evaluations. We start with improving the baseline performance of the deep network, then follow with comparisons to the state of the art.

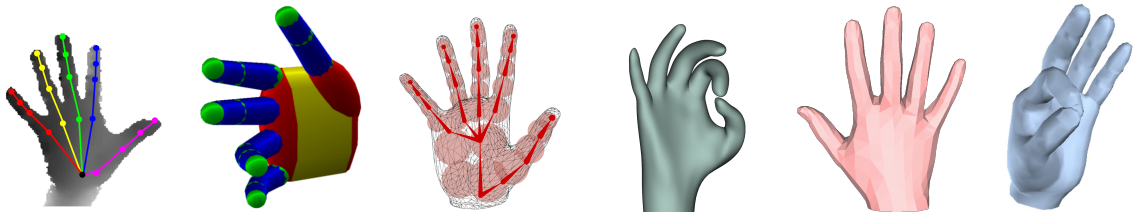
Chapter 6 presents the final conclusions of this thesis. We discuss caveats and future research to be undertaken.

2 Literature Review

We review the current literature in 3D hand pose estimation to make informed design decisions later on in chapter 3. We start by taking a look at how hands have been abstracted in the past. Following this, we examine the literature for depth and RGB based methods. The approaches can be divided loosely into 3 categories. Statistical methods relying on prior knowledge, iterative methods that iteratively update a hypothesis hand pose to minimize an error function and a hybrid of these two methods. Public datasets and benchmarks are surveyed to correctly evaluate the work in this thesis. In section 2.4 the core algorithms used in hand pose estimation are reviewed.

2.1 Hands

We first take a look at how hands have been represented in the past. The abstraction of a hand used determines the advantages and limitations of a given approach.



(a) (b) (c) (d) (e) (f)

Figure 2.1: Hand representations. (a) Skeleton Joints [3]. (b) Geometric *primitives* [4]. (c) *Sum-of-Gaussians* model [5]. (d) Smooth surface mesh [6]. (e) MANO model [2]. (f) Surface Mesh [7].

There are 3 general categories hands are divided into that we see: joints, parametric hands and surface mesh models.

The typical 21 joint skeleton, figure 2.1a, is represented by 21 3D locations (x, y, z) resulting in 63 Degrees of Freedom (DoF). Methods like [1, 9, 11, 20, 24] generate likelihood score maps for each of the joints in the hand. These score maps are subsequently used to regress a final hand articulation. A weakness of this detection based approach is

when inferring joints hidden from the camera. Without an inherent understanding of how the hand grasps objects, it becomes quite a challenge to infer joint locations that are hidden from view. [20] uses a kinematic skeleton to fit these 2D joint heat maps to produce only kinematically possible predictions.

An analysis on hand grasps demonstrates that hand poses live in a low dimensional space. [25] finds that the *first two principal components could account for > 80% of the (grasp) variance* when holding objects. [26] finds that full hand pose lives in ≈ 6.5 dimensions. Joint based hand methods live in a very high dimensional space compared to this, rudimentarily 63 dimensions without constraints. Parametric hand models are one abstraction that lives in a lower-dimensional space and encode kinematic constraints on hand articulations [2, 4, 5, 6].

Parametric hands are models that can be controlled via vectors of parameters. Compared to joint based hands, they can live in a lower-dimensional space and are generally restricted to kinematically possible articulations. A shortcoming is that they are typically static, not adjusting to the subject's hands. The MANO model is an exception to this.

2.1.1 MANO

We examine the MANO (hand **M**odel with **A**rticulated and **N**on-rigid def**O**rmations)[2] model in depth.

The MANO hand is a function of two vector parameters, pose $\vec{\theta}$ and shape $\vec{\beta}$ [2]. The formulation is originally from the SMPL [27] paper, where this method is used to parameterize a full body model. The model $M(\vec{\beta}, \vec{\theta})$ is defined as:

$$M(\vec{\beta}, \vec{\theta}) = W(T_P(\vec{\beta}, \vec{\theta}), J(\vec{\beta}), \vec{\theta}, \Psi) \tag{1}$$

$$T_P(\vec{\beta}, \vec{\theta}) = \hat{\tau} + B_S(\vec{\beta}) + B_P(\vec{\theta}) \tag{2}$$

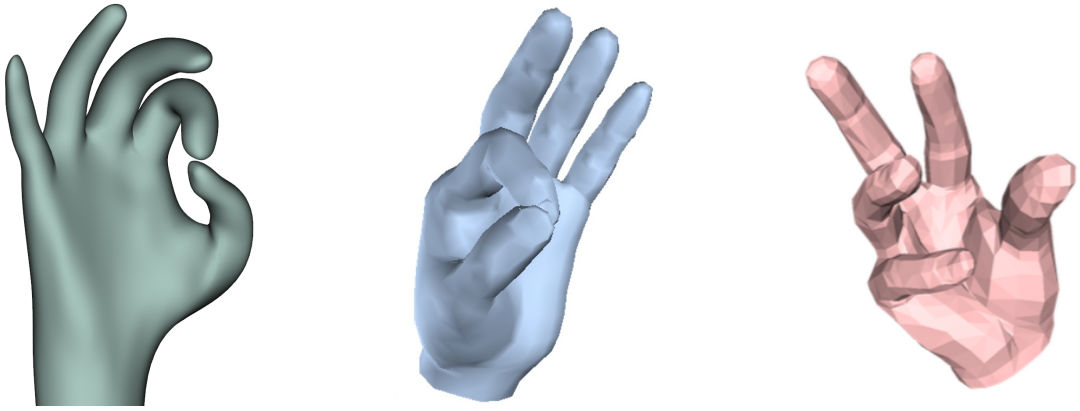
where the final hand model is built from a Linear Blend Skinning (LBS) function W applied to the *articulated rigged body mesh with shape* T_P, joint locations J defining the kinematic hand, pose $\vec{\theta}$ parameters, shape $\vec{\beta}$ parameters and blend weights Ψ [2]. Unlike other hand models that use LBS, the MANO mesh $T_P(\vec{\beta}, \vec{\theta})$ skinning function varies a base hand shape $\hat{\tau}$ with a blendshape function $B_S(\vec{\beta})$ that allows the hand to vary by skeleton shape, and a blendshape function $B_P(\vec{\theta})$ based on the pose parameters to correctly deform the skin around bent fingers.

What makes this parameterized hand model stand out from previous work [4, 6] is its construction. 2018 high resolution scans of 31 subject's hands were collected in various poses and grasping different objects. Left hand scans were mirrored to provide right hands,

Figure 2.2: Examples of some captured hands (*top row*) and the corresponding MANO hand *(bottom row)* for a single subject. From [2]

.

enabling Romero *et al.* to learn a single consistent hand model [2]. These scans capture the small nuances and deformations in hand poses with how fingers bend and the skin deforms under certain articulations. This construction technique results in a very real looking hand model.



(a) LBS over skeleton [6] (b) Surface Mesh Prediction [7] (c) MANO model [2]

Figure 2.3: Comparison to other hand skinning methods.

Two properties of the MANO model, in particular, make it stand out from other abstractions of the hand. An accurate skinning function with correct skin deformations that can be modified to fit a subject's hand, makes this parametric hand stand out from other developed parametric hands. Taylor *et al.* uses a hand skeleton and applies Linear Blend Skinning over that. The resulting surface mesh is recognizable as a hand, however, it lacks the correct skin surface deformations that the MANO model features. The underlying skeleton also can't be adjusted, the hand is locked with its current shape, figure 2.3 (a).

Ge *et al.* directly outputs a predicted surface mesh, figure 2.3 (b). This enables a perfect prediction of a user's hand shape. Their hand predictions, however, lack a kinematic structure. The second property that makes this MANO model stand out from non-parametric hand representations is that it lives in a lower dimensional space and is kinematically constrained. The complexity of generating an accurate surface mesh outside of its training data proves to be a challenge in the approach Ge *et al.* proposes, figure 3.2. The model is highly dimensional, losing the benefits that a parametric hand can provide, as discussed in section 3.1.

We can see, per figure 2.3 (c), the accurate surface mesh the MANO model produces with correct skin deformations.

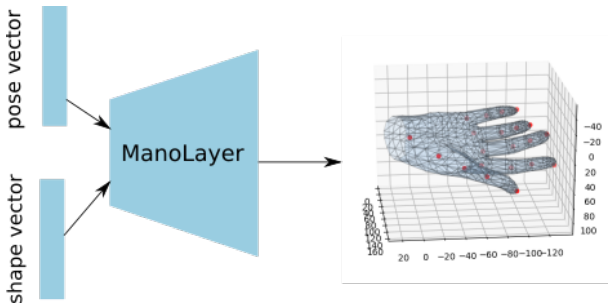


Figure 2.4: MANO model function.

Using the MANO model in practice is quite straightforward. Passing in vectors of pose $\vec{\theta}$ parameters, $\theta \epsilon \mathbb{R}^{45}$ and shape $\vec{\beta}$ parameters, $\beta \epsilon \mathbb{R}^{10}$, the MANO model returns an accurate hand mesh with corresponding skeleton joints, figure 2.4

2.2 3D Hand Pose Estimation

We focus on markerless monocular based methods. Other methods include marker and glove based tracking, or multi-camera/stereo tracking setups. We are interested in monocular based tracking for AR/VR and other applications of hands-free human-computer interaction.

A review of depth-based State of the Art (SOTA) hand pose estimation finds that *statistical methods still generalize poorly to unseen hand shapes* [28]. It was found by [29] that *a simple nearest-neighbour baseline outperforms most existing systems. This implies that most systems do not generalize beyond their training sets*. Evident from the highly accurate benchmark results from current methods like [7, 9, 10, 12] achieving $\approx 10mm$ of mean joint error, it appears that a big challenge in hand pose estimation is correctly generalizing outside of training data to produce reliable predictions. These systems are capable of achieving strong accuracy on benchmarks. They now need to be capable of producing these levels of accuracy outside of their prior knowledge.

It is quite common to see multiple-stage pipelines for hand tracking. Various methods de-compose the problem of recovering a 3D pose into multiple steps. Common multiple-stage pipelines we have seen first predict 2D joint locations with one network, and then regress 3D joints locations from the 2D joints [1, 9, 20]. Other methods first generate an initial hand hypothesis and iteratively update their prediction based on an energy function [6, 8]. We have also seen methods that augment and synthesize data in the middle of the pipeline to improve prediction accuracy [10, 24].

2.2.1 Depth-based approaches

Majority of the recent work in hand tracking has been done on depth imaging, in part due to the *2017 Hands in the Million* [29] and the *Hands19*[3] challenges.

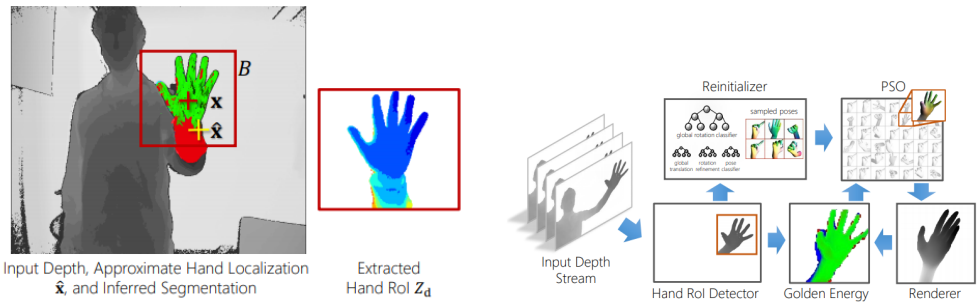
Depth images are simply represented by 2D arrays consisting of depth values $Z = \{z_{ij} \mid 0 \leq H, 0 \leq W\}$, where z_{ij} is the depth value stored at pixel (i, j) and H and W is the image height and width respectively.

The nature of depth maps containing 3D information provides for some innovative techniques. We see methods like [8, 10, 22, 24] using depth maps as 2D input images. However, we also see methods like [30], [9], [31] convert the 2D depth maps into 3D formats before feeding them into 3D CNN's to produce 3D predictions from 3D input data.

Iterative

Iterative methods like [8] and [6] use a pre-defined hand model to search for a pose that best fits the input depth image. These methods search a pose space iteratively for better hypotheses that fit a given depth image based on a fitting/loss function.

Iterative methods use stochastic algorithms like Particle Swarm Optimization or Stochastic Gradient Descent to update a hypothesis pose iteratively based on a given heuristic. These methods are not dependant on prior knowledge like statistical models. We still see training data and prior knowledge used by these methods to optimize the search strategy. They can be computationally expensive to compute in real-time, requiring multiple iterations to converge to an acceptable global minimum.



Hand RoI extraction Overview of the pipeline

Figure 2.5: Iterative PSO model based method. A hand pose is iteratively updated based on loss function 3. From [8].

We take a deeper look at two iterative methods. [8] extract a hand Region of Interest (RoI) from the input depth image, Z_{roi}. An initial hypothesized hand pose is selected and a synthetic depth image of that hand is rendered, R_{roi}. A loss or *"Golden Energy"* function, is

calculated based on a pixel-wise difference between the rendered hand R_{roi} and the extracted input Z_{roi}.

$$E^{Au}(Z_{roi}, R_{roi} = \sum_{ij} \rho(z_{ij} - r_{ij})) \tag{3}$$

Where ρ is a truncated distance metric, $\rho(e) = min(|e|, \tau)$. This golden energy function is used to guide an optimizer based on particle swarm optimization, where each of the hypothesized hand poses is a 'particle' in the pose population.

Taylor *et al.* has a similar approach [6]. The energy function is the key determining factor in these approaches. Where Sharp *et al.* use a rendering and golden energy function [8], Taylor *et al.* use a weighted sum of several terms. The used terms dictate that the hand model hypothesis should line-up with the surface points in the depth image, the hand pose should be a likely hand pose and the hypothesis should be kinematically correct, without any self-intersections or broken geometry.

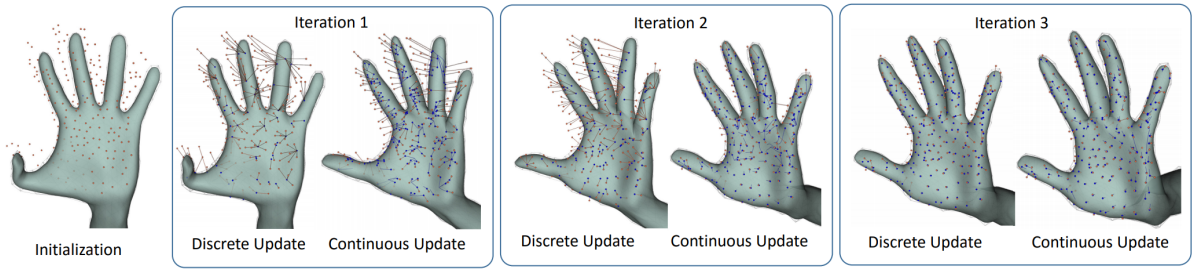


Figure 2.6: Taylor *et al.* fitting a model based on depth surface points. From [6].

Since the hypothesized poses are driven by parameterized hand models, kinematically impossible poses are intrinsically not included in the search space.

Statistical

Statistically based methods attempt to regress hand poses straight from input images based on training data. Methods like [9, 10, 24, 30, 31] train various deep networks to learn and regress 3D joint locations.

Statistical methods commonly used are the typical statistical machine learning models such as Random forests [32], nearest-neighbours [8], and Convolutional Neural Networks (CNN) [10, 12]. They use prior knowledge in the form of training data to learn statistical distributions of hand poses based on a given input.

We look more closely at [9] which won the Hands 17 challenge [29] and [10] which won the Hands 19 depth challenge [3].

[9] take advantage of the 3D information a depth map contains. They argue that regressing 3D coordinates from a 2D CNN has a few weaknesses. Firstly, perspective can easily cause distortion in the 2D map. Secondly, a regression from 2D space to 3D coordinates is a highly non-linear mapping, making learning more difficult.

We notice that their method is similar to that of [11], but in 3D space. [11] generate per joint pixel-wise heatmaps. [9] overcome the weaknesses they identify from 2D by transforming their network and depth maps into a 3D space. The 2D depth maps are converted into 3D voxel spaces which are fed into a 3D CNN. The per voxel likelihoods of each key joint is generated from this network and a hand pose is regressed from these joint locations.

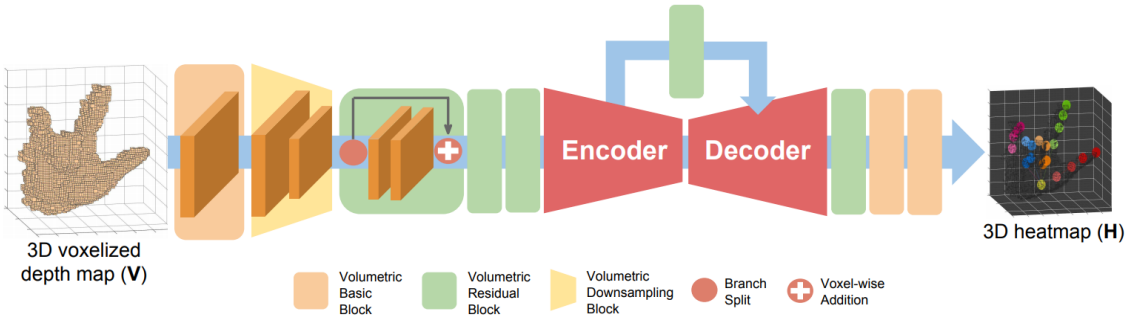


Figure 2.7: V2V-PoseNet. The network converts a depth map to a 3D voxelised format. This format is fed into a 3 dimensional network which outputs a *per-voxel likelihood for each keypoint.* [9]

In contrast to [9], [10] proves these 2D to 3D predictions possible. [10] improves the SOTA results of [9] without the use of 3D networks. They find the 3D CNN harder to train and computationally expensive and based on their results, 2D CNN's evidently produce sufficiently accurate predictions.

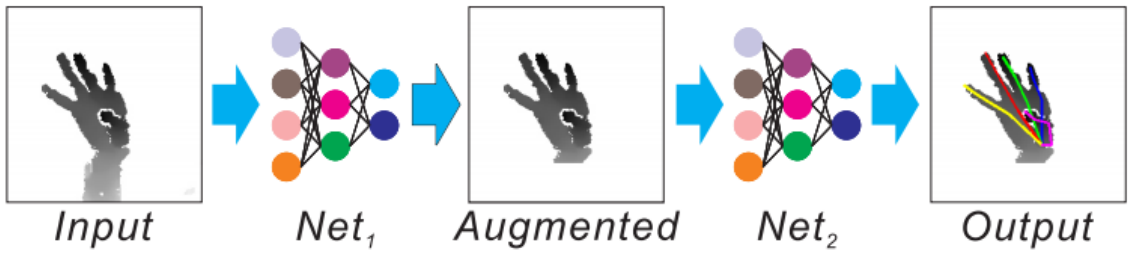


Figure 2.8: Overview of the two stage network [10].

Their solution involves two stages [10]. A first network takes in a depth image and outputs an augmented, cleaner image, which is subsequently fed into the second network. The second network then directly regresses the 21 joint locations.

The work of [10] proves to us that simpler 2D networks are capable of learning to map from 2D input information to 3D coordinates.

Hybrid

Iterative approaches tend to be computationally slower than a direct inference from a deep net. It can take quite a few calculative iterations to generate a satisfactory hypothesis pose. Statistical approaches we have seen evaluate strongly in benchmarks. In practice, they over-fit their training data and do not generalize well, [28], figure 3.2. Kinematic and physical constraints are typically lost with the statistical approaches [1, 7].

Hybrid methods make use of both these aforementioned techniques. They use a statistical method to initialize an iterative search method with a strong hypothesis. This reduces the time taken to converge, utilizing the advantages both approaches offer.

Hybrid methods combine iterative and statistical approaches in an attempt to overcome each other's weaknesses. In practice, we see a statistical approach generate an initial set of hypothesis to create a much narrower search space for an iterative method to search through. [33, 34, 35, 36]

[36] uses a CNN to initialize PSO with predictions. [8] does the same with their optimizer, instead opting to use a decision-forest to optimize their search space. [6] uses a mix of a hypotheses from a retrieval forest algorithm [37] and the pose predictions from previous frames in a video sequence.

2.2.2 RGB

RGB images are only 2D and lack the depth information that enables intrinsically 3D iterative methods mentioned in 2.2.1. RGB imaging poses a naturally tougher challenge according to [38] [16] in that we are attempting to map from 2D input information into a 3D space. The lack of 3D information in RGB images also means we do not see iterative type methods as seen in section 2.2.1, we have to rely on more traditional vision-based methods. Deep networks are generally the only methods we see capable of mapping from 2D RGB imaging to a very large 3D pose space in current literature.

More traditional detection-based methods have been shown to work when it comes to 2D pose recognition [11]. 3D pose estimation literature uses these 2D poses as input to deep networks to regress 3D poses [1, 7, 24].

[16] and [1] argue that making a 3D prediction from a 2D image is *ill-posed*. We have seen [9] claim that learning a 2D $\rightarrow$ 3D mapping is much harder due to its *highly non-linear mapping*.

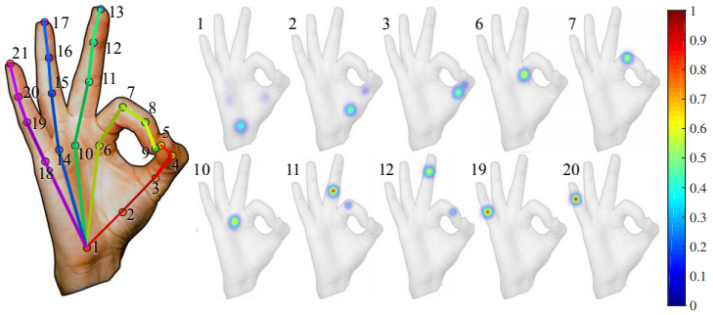
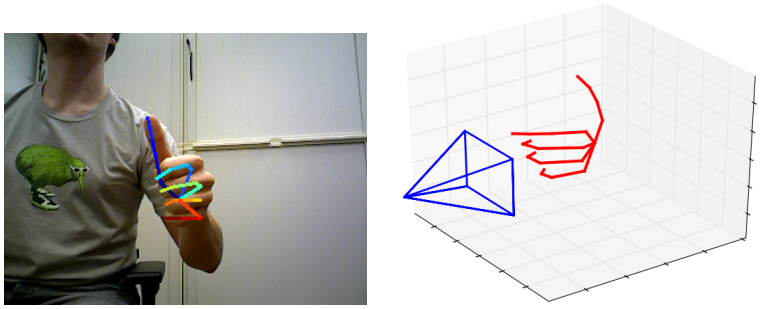


Figure 2.9: 2D pose is recovered from score maps for each joint location. From [11].

Many methods, rather than predicting 3D joint locations from a 2D image, instead produce 2D joint locations from a 2D image. Then they regress a final 3D joint prediction from the 2D joints [1, 7, 20].



(a) (b)

Figure 2.10: Zimmerman *et al.* lifting 2D keypoints to a 3D pose. (a) 2D joints blended onto input image. (b) Joints projected into 3D space. From [1].

Zimmerman *et al.*[1] pose a two-part network solution. They use their first CNN to generate 21 score maps for each of the joint locations. Subsequently, these 2D score maps of the joint locations are used as input for a second CNN whose job it is to produce the most likely 3D locations.

GANerated hands [20] follows a similar structure to the approach of Zimmerman *et al.* [1]. They generate 21 heatmaps for 2D joints, using these heatmaps as features for a skeleton fitting process. Fitting this parametric hand skeleton results in kinematically correct hand predictions. They combine this approach with a dataset generated with a Generative Adversarial Network for particularly real looking synthetic images.

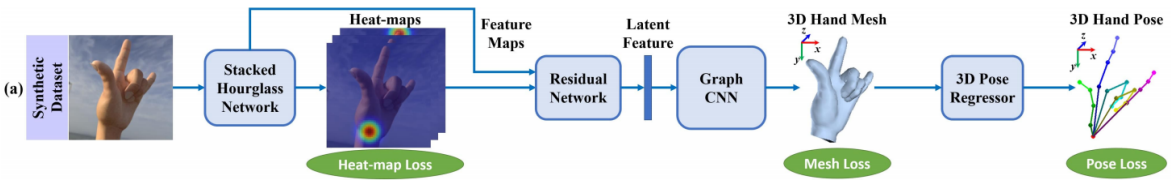


Figure 2.11: Generating surface mesh with a Graph CNN. From [7].

[7] propose a *Graph Convolutional Neural Network (Graph CNN) based method to reconstruct a full 3D mesh.* Heat maps of keypoint locations are once again generated and fed subsequently further into the network. Ge *et al.* recognize meshes can be represented as graphs, and thus built a graph-based CNN network which directly outputs a recovered hand mesh of what is present in the RGB image. Joint locations can consequently be recovered from this hand mesh using a linear regressor, giving us both hand pose and shape. The method can be quite computationally heavy with a large network that outputs 1280 mesh vertices.

[12] attempts to tackle hand pose estimation under occlusion during interaction with objects. Firstly, they use the parameterized MANO hand model [2] from section 2.1.1. Use of this hand model allows recovery of hand-shape through the shape vector $\vec{\beta}$ as well as pose articulation from the pose $\vec{\theta}$. Hasson *et al.* use two separate networks, a hand branch and an object branch, to output a single hypothesis on the hand parameters and to predict a rough object mesh.

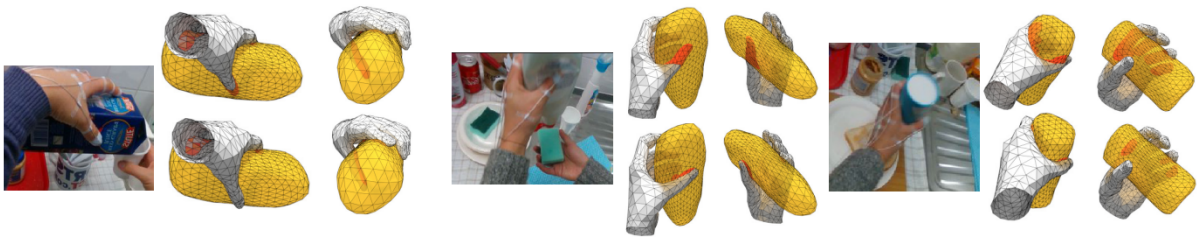


Figure 2.12: Qualitative example of [12] generating hand poses *without* (top) and *with* (bottom) contact loss on the [13] dataset. We can observe the meshes intersect (highlighted in red) a lot less, producing a more realistic prediction. Object predicted mesh is in yellow. From [12].

With a hand and object mesh, Hasson *et al.* iteratively update pose predictions through a contact loss function, to correctly articulate the hand to grasp the object mesh. A physical simulation of the meshes is also run for a few iterations to ensure a stable hand grasp of the object is achieved. This method relies on an object being grasped by a hand, however, the key observation here is that hand pose can be inferred from the object that is being grasped. There are only so many reasonable configurations a hand can grasp an object. Hasson *et al.* use the predicted object meshes to produce realistic hand grasps.

RGB-D based methods also exist, using both RGB and Depth data. Consumers depth cameras, such as Intel Realsense D435 and the Microsoft Kinect 2, both have RGB cameras as well as their depth modules. Methods like [39] [40] use depth information as privileged information during training. Privileged information is information available to the models only when learning. The privileged depth information helped their models learn more accurate pose estimation functions on RGB data. We have also seen [7] use privileged depth information as a lightly-supervised method of training their models on real data. Rather

than having to manually annotate RGB images with surface hand meshes, they used the depth images to produce reasonable estimates of the surface mesh.

2.3 Datasets

Deep learning is very dependent on the data we feed our networks. The data publicly available for this research largely influences what can be accomplished.

It is important to also understand how ground truths have been annotated. Manually annotating 2D colour images with 3D joint locations is impossible. Thus we do not consider colour datasets that only provide 2D joint annotations. 3D joint annotations is a requirement for training a 3D hand pose estimator.

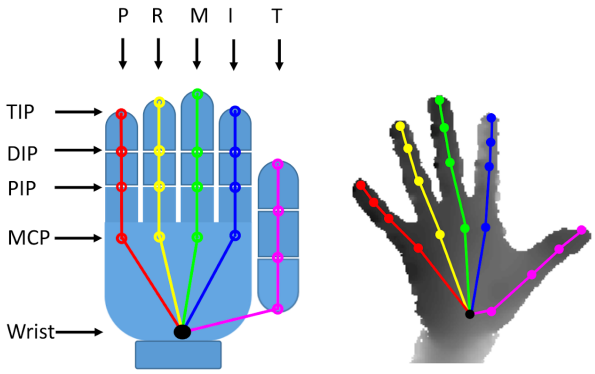


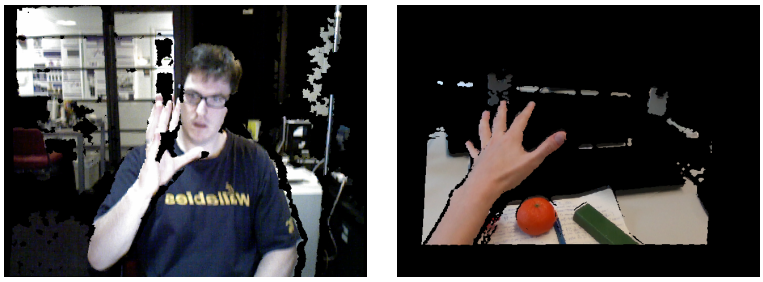
Figure 2.13: Datasets are typically annotated with 21 joints. Each dataset would provide details on which finger joints appear in what order in the annotations. From [3].

Not all datasets are annotated with this standard, which is something to look out for. ICVL[18] is annotated only with 16 joints, while NYU[19] is annotated with 36 joints. The NYU[19] dataset could still be used for a 21 joint training model if you only use the 21 annotated joints that are relevant. The EgoDexter[41] is only annotated with the 5 fingertips on depth information. ICVL and EgoDexter being annotated with fewer than 21 joints makes the datasets useable for validation and benchmarking, but would not be ideal for training.

Some datasets like NYU[19] annotate their data by first using a pre-existing iterative method to fit a hand model to input depth information. NYU[19], STB[15] and HO-3D[42] use multi-camera capture systems to capture hand poses. 3D annotations can then be made from these multiple-viewpoints without the use of any kind of markers. EgoDexter[41] is manually annotated on the depth information at the fingertips. FHAD[13] uses magnetic sensors on the hand to automatically infer the 21 joint locations of the hand. These magnetic sensors are stuck to the hand, however, polluting the colour information. They are flat enough to not distort the depth information.

Synthetic datasets are also extremely common. They give the advantage of having perfect ground truth annotations and lots of data can be generated with all kinds of hand poses and shapes. Synthetic datasets do come with the disadvantage of potentially not looking real, so it is important to also examine the realism of these synthetic data generation methods. Realistic synthetic depth data is easier to generate than colour.

Certain datasets suit different methods based on how they have been built. For example, RHD[1], FHAD[13] and NYU[19] do not feature hands that have already been cropped and centred. Most of the methods we discussed in section 2.2 would assume pre-processing to isolate, crop and centre the hand to have already been done.



(a) NYU [19] (b) EgoDexter [41]

Figure 2.14: Examples of registered colour images, which only provide colour information on pixels that have depth information.

Datasets like NYU[19] and EgoDexter[41] feature mixed RGB/D data. However, colour data is only provided on pixels with valid depth data. This produces fragmented colour images not suitable for RGB only methods, figure 2.14.

The number of subjects in a dataset is also important. We want our training data to match the real-world distribution of hands as much as possible. The FHAD[13] dataset, for example, features poses and actions from 6 different subjects. This gives our models an opportunity to learn different hand shapes as well as poses. The FreiHand[43] dataset is even annotated with exact hand meshes, allowing us to properly learn hand shapes from a mesh loss function.

Viewpoints and occlusions are two other factors to be considered in a dataset. Poses from the first and third person perspectives look very different, with egocentric views suffering in particular to self occlusions. FHAD[13] and EgoDexter[41] feature hands in an egocentric viewpoint interacting with objects. Obman[12] and HO-3D[42] feature hands interacting with objects from the 3rd person perspective. When hands are interacting with objects, it is useful to know what kind of annotations are provided for the object. Methods like [12] use a network to predict the 6D pose and mesh of an object to refine their hand prediction. Some sequences in FHAD[13] have the objects wireframe-shape and pose annotated.

We summarize some relevant datasets to 3D pose estimation in recent years, table 2.1.

Dataset name	Modality RGB/D	Type Real/Synth	Viewpoint 1st/3rd	Objects present?	# Subject hands	# Frames Train/Test	Year created
NYU[19]	Depth	Real	3rd	No	2	72k/8k	2014
ICVL [18]	Depth	Real	3rd	No	10	331k/1.5k	2014
FHAD [13]	RGB+D	Real	1st	Yes	6	100k	2018
EgoDexter [41]	RGB+D	Real	1st	Yes	4	1485	2017
SynthHands [44]	RGB+D	Synth	1st	Both	2	63530	2017
BigHand2.2m [17]	Depth	Synth	3rd	No	10	2.2 million	2017
FreiHand [43]	RGB	Real	3rd	Yes	N/A	130k/3960	2019
RHD [1]	RGB+D	Synth	3rd	No	20	41k/2.7k	2017
GANerated [20]	RGB+D	Synth	1st	Both	N/A	330k	2018
STB [15]	RGB+D	Real	3rd	No	1	18k	2017
Obman [12]	RGB+D	Synth	3rd	Both	N/A	21k	2019
HO-3D [42]	RGB+D	Real	3rd	Yes	-	80k	2019

Table 2.1: Summary of public hand datasets (non-exhaustive).

All synthetic datasets inherently are annotated with accurate ground truth annotations.

For real datasets, RGB based modalities commonly use a stereoscopic camera setup to recover 3D joint locations from 2 different 2D images [15, 42, 43]. Depth-based datasets can be manually annotated in 3D space by annotators on a single image due to the 3D depth information. Most RGB based datasets use RGB-D cameras to utilize the depth information to produce accurate ground truth annotations. The depth information is then also presented with the dataset as optional extra data [13, 15, 41, 42]. Synthetic RGB datasets typically also contain depth information, as it just requires passing the 3D hand through a depth renderer [1, 12, 44].

Figure 2.15: Magnetic sensor annotation method. From [13].

FHAD [13] uses thin magnetic sensors attached to subjects to regress joint locations, figure 2.15. The dataset was captured in RGB and Depth formats. The magnetic sensors are thin enough to not interfere with the depth data, which does not pick them up [13]. It does, however, pollute the RGB data. We do not know if a system will become dependent on detecting the magnetic sensors and thus fail in actual use.

2.4 Algorithms

We briefly take a look at the fundamental techniques of hand pose estimation used in current literature.

Random Forest (RF)

Random forests have shown good results in older literature from 2011-2015 for hand pose estimation from depth images. [8, 18, 32].

Random forests are an ensemble learning method. For training, multiple variations of decision trees are constructed. For inferencing, the modal prediction from the trees is used in classification tasks and the mean of the predictions for regression tasks.

[8] creates buckets of 127 global rotation buckets, using a random forest to classify global rotation in the first part of a multi-stage process. Random forests are what were used in the initial release of the Kinect [45].

Particle Swarm Optimization (PSO)

Particle Swarm Optimization (PSO) is a stochastic optimization technique for non-linear functions [46]. We have seen it successfully used in hand pose estimation as early as 2011 with [4].

PSO iteratively updates a hypothesis hand configuration, minimizing a difference heuristic from a given input. This algorithm is prominent through depth-based tracking methods [6, 8, 47]

PSO consists of a population (swarm) of individual entities (particles) that each traverse the problem space. Particles iteratively update their positions based on their own best-known positions, as well as other particle's best-known positions. This enables the swarm to share information between particles, helping escape local minima and maxima.

Despite this, due to the high dimensionality of the pose articulation space, and the non-convex nature of the function we need to learn, it is still possible for PSO to get stuck in local minima. [8] uses random forests to initially generate a pose hypothesis, close to a global minimum, improving the chances for PSO to find the global minima.

Deep Learning

Most research published in the recent years primarily uses deep learning, specifically Convolutional Neural Networks (CNN's) and its many architectures. CNN's are a branch of artificial neural networks that use a convolution mathematical operation on a given input,

image matrices in our case. These CNN's produce vectors of image features, which can then be fed into a small fully connected neural network to produce final predictions.

With proper training deep networks are more capable of learning t he highly dimensional function mappings necessary for hand pose estimation compared to more traditional machine learning methods like PSO and RF's.

RF and PSO depend on heuristics for hand pose accuracy by which they iteratively update their hypotheses. Depth imaging with its 3D properties enabled these algorithms. We have not noticed RGB based heuristics in the literature reviewed that enabled these algorithms. In the early years for hand pose estimation *circa* 2011 to *circa* 2015, before deep learning, we mostly only observe depth-based research. Since then we have seen deep learning dominate hand pose estimation, enabling impressive RGB based approaches [1, 7, 12].

We have seen deep learning used a lot more now in depth-based methods in recent literature [10, 24]. 3D deep networks have also been used with leading results, outperforming their predecessors. [9, 30, 31]. All the reviewed 3D RGB based literature in this work uses deep learning to regress pose estimations [1, 7, 12, 20].

The ImageNet challenge demonstrates the progress deep learning has made in past years [48]. ImageNet is a large-scale image database featuring annotated objects. The challenge is famously used for pushing the SOTA of CNN architectures by measuring their classification accuracy against this dataset. AlexNet [49], VGG [50], GoogleNet [51] and ResNet [52] are architectures that have previously won this challenge. The human classification error rate is approximately 5.1% on the ImageNet dataset [23]. ResNet surpassed this error rate in 2015 for the first time with a classification error rate of $\approx 3.57\%$ [52].

Network architectures got deeper each year in the ImageNet challenge since AlexNet came out with 5 layers. VGG came out later with 19 layers and GoogleNet subsequently with 22 layers. As deep networks got deeper, the vanishing gradient problem was exacerbated. Weights in a neural network are iteratively updated based on the partial derivative of an error function that is back-propagated through the network layers. As the gradient is back-propagated through more layers, the gradient may come very close to 0, effectively freezing the weights and preventing a network from learning any further.

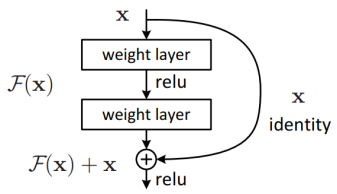


Figure 2.16: A residual block. From [52].

The ResNet architecture that achieved the breakthrough accuracy results in 2015, surpassing

human performance, was 152 layers deep. ResNet, a residual network, is built out of residual blocks, figure 2.16. A residual block is just made up of two, sometimes three, normal CNN layers where the identity of x, the input to the layer, is also skipped forward and added with the output of the skipped layers $\mathcal{F}(x)$. Skipping layers in these residual blocks reduces the impact of vanishing gradients because there are fewer layers to back-propagate through, which is how this network is successfully much deeper than previous ImageNet winners. We see this network architecture commonly used in the recent literature for hand pose estimation [7, 9, 12, 20].

3 Design

In this chapter we discuss the shortcomings seen in the literature from chapter 2 and why we chose to use the MANO model.

There is a lack of research on how approaches to hand pose estimation can generalize outside of their training data. A review of depth-based methods found that *statistical methods still generalize poorly to unseen hand shapes* [28]. Figure 3.1 demonstrates a saturation in benchmark performance for various methods. Benchmarks reward over-fitting and currently, we do not see enough research on how these various methods can track hands outside their training data or under object occlusion.

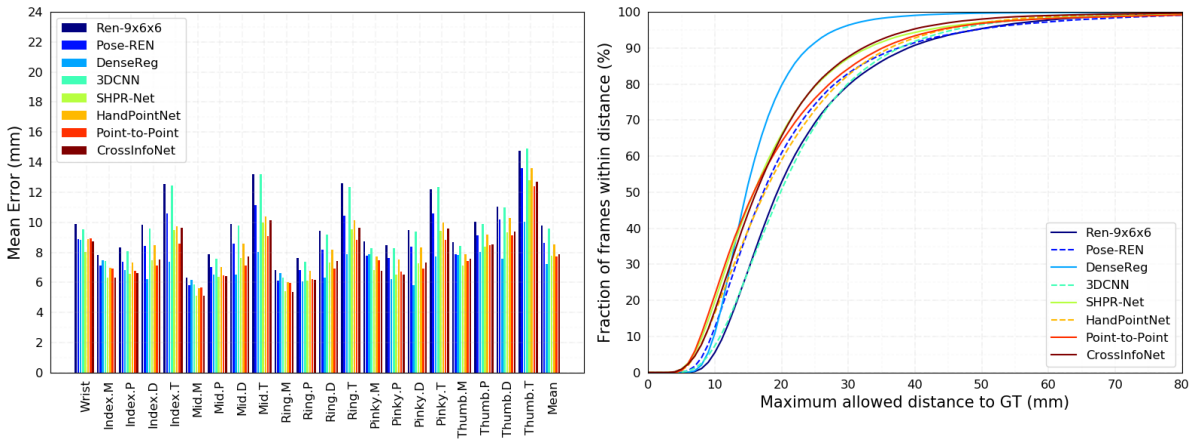
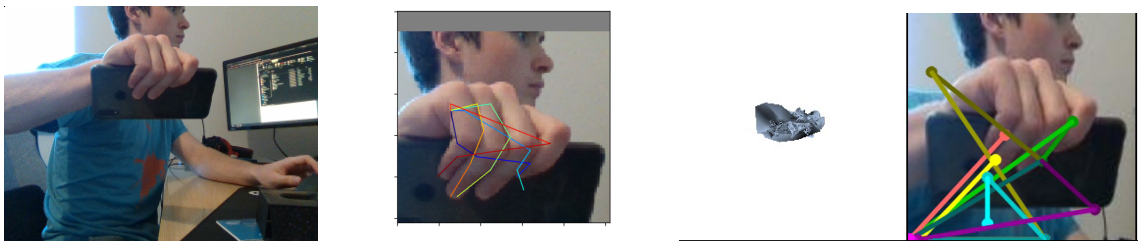


Figure 3.1: MSRA dataset evaluations for SOTA methods. From [14].

Heuristic fitting methods like [6, 8, 9, 31] assume isolated hands. Hands interacting with objects would throw off their fitting functions. We tried out two RGB methods on a sample image outside their training data under object occlusion, figure 3.2. Their predictions on this particular image failed to produce a hand at all.

We chose to evaluate the MANO model approach as we think it can potentially enable a deep network to generalize outside of its training data while being comparatively accurate to current RGB methods and still track hands under occlusion.



Input Image (a) Hand3d Output [1] (b) HandMesh output [7]

Figure 3.2: Testing two RGB methods outside their training data under occlusion.

3.1 MANO based pose estimation

To correctly infer a pose from a hand under object occlusion, a hand pose estimation method needs to be able to correctly infer joints that are hidden from view based on the visible features of the hand. This requires the network to have an understanding of the hand, by means of constraints [1], or by using hand model [6, 8, 12, 20]. Without this kinematic constraint on output predictions, outputs like figure 3.2 (b), that do not resemble a hand, occur. We want to use a hand model somewhere in our approach to constrain predictions to physically correct hands and potentially help interpolate hidden features of the hand as claimed by [20].

Analysis on hand pose finds that hands live in a low dimensional space[25][26][53], between 2.5 and 6 dimensions. The representation of hands in pose estimation literature thus far has been quite high in comparison, ≈ 45 with various parametric hand and joint representations. By lowering the dimensionality of the hand pose estimation problem, and constraining predictions to kinematically correct hands, pose predictions should generalize better outside of training data compared to approaches in a higher pose space that do not utilise a parametric hand or an equally low dimensional hand abstraction.

We chose to use the MANO model as the parametric hand abstraction in this research per comparison to other models in section 2.1.1. Typically, a network outputs a set of joints or surface mesh vertices, requiring a function $\Theta : \mathbb{R}^{256x256} \mapsto \mathbb{R}^{21x3}$ to be learned. The MANO model can be treated as a function of 1-dimensional input vector $M : \mathbb{R}^{45} \mapsto \mathbb{R}^{21x3}$. Functionally composing a network function ρ with the MANO model M, $\rho \circ M \equiv \Theta$, reduces the learning function Θ to $\rho : \mathbb{R}^{256x256} \mapsto \mathbb{R}^{45}$.

The MANO model abstracts the 3D inference from the network. The network does not have to learn to predict 3D locations from a 2D image. Instead, it learns a 2D $\mapsto$ 1D mapping for the $\vec{\theta}$ MANO pose parameters.

The creators of the MANO model also use Principle Component Analysis (PCA) to further reduce the size of the prediction vector, pushing the dimensionality of it even lower. [12] is a hand-pose object estimator that makes pose predictions of hands interacting with objects

and also experiments with PCA. We hypothesize that there is typically enough information on partially obscured hands to still produce reasonable pose predictions given the dimensionality of the function a deep network learns is low enough.

Other parametric hand methods typically use their hand models to minimize an error[6, 8, 20, 54]. We want to avoid fitting an energy function for the circumstance of hands interacting with objects. Hands and how people use them are unpredictable. We do not foresee reliable pose predictions in situations that an energy function is not designed for.

Method	Inference Time (seconds)	Frames Per Second (FPS)
Obman (2019)[12]	0.06	16.6
Hand3d (2015)[1]	19.6	0.05
HandMesh (2019)[7]	0.24	4

Table 3.1: Real-time performance of various methods on the benchmark machine.

Many methods use multiple stages in a pipeline and multiple deep networks for pose predictions [1, 7, 8, 10, 12, 20, 24]. Table 3.1 shows the runtime performance of some methods on the benchmark machine for this research, specs given in table 4.1. On more reasonable consumer hardware these methods do not run fast enough for real-time hand pose estimation. We use a single deep network to infer hand pose in one pass. We do this to be more efficient for real-time performance. The single network is also simpler and quicker to train than other multiple-network based approaches, enabling training on a wider variety of data.

In the absence of a heuristic fitting function, and wanting to be as efficient as possible for real-time performance, the initial pose prediction from the network is taken as the final prediction. There are no further pose optimization steps taken to improve the accuracy of the prediction from the deep network. The evaluations of this approach will demonstrate the ability for a deep network to mimic hand pose in images without additional optimization.

Deep learning, specifically CNN's, have made great progress in the last few years, recently out-performing humans in object classification tasks. [23] found human classification error to be $\approx 5.1\%$ on the ImageNet dataset. ResNet surpassed this error in 2015 for the first time with a classification error rate of $\approx 3.57\%$ [52]. The Residual network architectures have demonstrated success in other approaches for hand pose estimation [7, 9, 12, 20]. The choice of the deep network architecture was not a focal point in this research. We just chose ResNet as that architecture as it has already been validated to work in hand pose estimation. We would expect that as deep learning improves, this approach will improve accordingly.

We present the implementation of this approach in chapter 4.

3.2 Modality (RGB vs Depth)

Depth-based cameras initially enabled robust commercially available body pose estimation systems following the Kinect in 2011 [45]. Depth-based information has enabled traditional machine learning techniques like random forests to predict reliable pose estimations from as early as 2011 [18, 32, 45]. We have also seen a majority of recent research focusing on depth-based methods from challenges like the Hands in a Million [28] and Hands19 workshops [3]. The 3D information a depth image allowed researchers to overcome the weaker deep learning models of the time. For example, [6, 8], use random forests to generate initial hand hypothesis and iteratively search the nearest neighbours for a hand model that better fits the 3D information presented in the depth image, which is not possible with RGB.

We have seen a lot of progress in deep networks in the previous years. Networks designed in the ILSVRC [48] are now outperforming humans in object classification. CNN's are now capable of learning some rather impressive tasks.

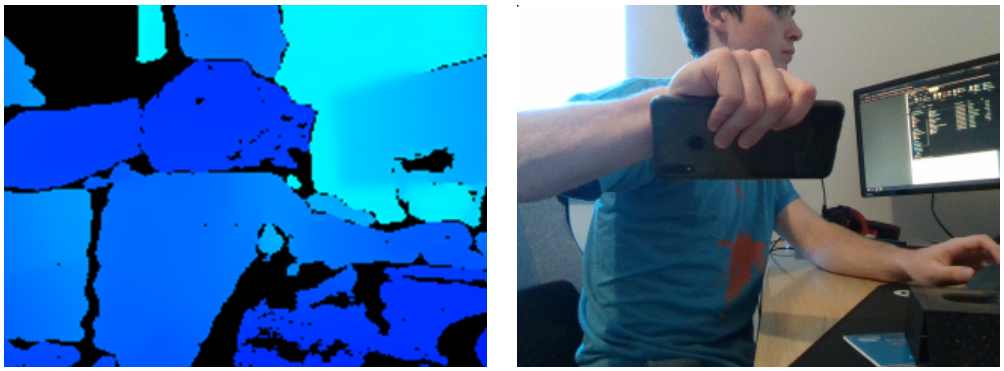
Depth RGB

Figure 3.3: Demonstrating "webbing" effect in depth data.

Depth cameras produce much lower resolution imaging at lower frame-rates compared to RGB modules. Kinect and RealSense cameras record depth up to 640x480 at 30 frames per second. Higher resolutions are achievable but require a lower frame rate, too low for smooth real-time tracking. Lower resolution negatively affects hand pose estimation in a few ways. Firstly, subjects must be captured closer up to the camera. My hand shown in figure 3.3 is $\approx 75cm$ away from the sensor. Already at this distance, the information density of the depth image is not enough to see the individual fingers, causing a "webbing" effect on my hand, figure 3.3. [24] overcome these shortcomings through data augmentation. They render an initial hypothesis into a synthetic depth image. Then they fuse the synthetic image with the original image, resulting in a sharper input to their pose prediction network. Body pose

estimation has been able to perform well with depth, given bodies are much larger, with features much simpler to distinguish. Hand pose estimation tends to struggle more due to higher degrees of freedom in the hand and more subtlety in hand articulations that depth sensors may not pick up.

As a result of lower depth resolutions, depth-based methods demand subject's hands be between $\approx 30cm$ and $1m$ from the camera. This would rule out many applications for controlling TV devices and projected screens.



Depth RGB

Figure 3.4: Comparing effect of occlusion between Depth and RGB.

Hand-object classification is also a lot tougher under depth. Even humans would have a very tough time correctly classifying the object being held in figure 3.4. The reduced amount of information a depth image contains compared to RGB makes hand and object segmentation quite challenging. Some depth cameras also cannot be used outdoors [55]. Sunlight, for example, interferes with the infrared module on the Kinect.

Depth-based tracking methods have certainly performed well thus far despite the issues. The 3D information a depth image inherently contains has been easier to teach computers to understand. Current depth camera modules do inhibit the potential use-cases and applications of a hand-computer interaction system.

In 2019, [10] outperformed the state of the art methods in the Hands19 challenge[3]. The authors simply augment input data by synthesizing synthetic images with the input data and feeding those augmented images into a 2D CNN, demonstrating that predictions from 2D data can outperform 3D based methods utilising depth. We do not expect the 2D nature of RGB images to inhibit the potential for 3D pose estimation.

RGB does contain depth information in the form of lighting cues, subject scale, occlusion and more. It has been more challenging to exploit the information RGB contains. We think RGB has more potential than depth for pose estimation in the future, so we apply our research to RGB images.

3.3 Datasets and benchmarks

We present the datasets used to experiment on this MANO based approach to test our research hypotheses.

We chose the Stereo-Based hand tracking Benchmark (STB) to train and evaluate this approach for accuracy on real data. The images were captured on an Intel Real Sense F200 depth camera. The dataset includes both RGB and Depth data. We hoped to also benchmark our performance using this depth information, but it ended up being out of scope for this research. This dataset is one of the commonly benchmarked datasets [1, 7, 12, 20], so there are many approaches to compare against. The dataset takes advantage of a stereo RGB-D camera setup to automatically and accurately annotate the ground truths values. We looked for annotation methods that were not manually labelled to try and avoid bias. Dataset information can also be found in table 2.1.

(a) Counting (b) Random pose (c) Varying Background

Figure 3.5: Samples from the Stereo Tracking Benchmark (STB) hand dataset. From [15].

The dataset features one subject going through two sequences of hand poses, counting numbers and random poses, against 8 different backgrounds, figure 3.5. We give details on the metrics evaluated in section 4.4.

We have seen many other methods benefit from using synthetic data to supplement training, particularly depth data [1, 7, 8, 17, 20, 24]. From figure 3.6 we can see synthetic datasets cover a greater range of hand articulations and viewpoints compared to real datasets. Annotating and capturing real datasets is a time consuming and laborious process. Ground truth annotations may be inaccurate based on the annotation method. Depth-based pose estimation research often uses synthetic datasets to supplement and generate large amounts of accurately annotated data very quickly. Synthetic depth-based datasets can be made to look very real, it is harder to generate synthetic RGB images that mimic the real world. We want to confirm that synthetic data can aid the deep network in learning how to mimic hand poses in real images.

We are using the Obman dataset to evaluate how the network benefits from synthetic data. Obman is a synthetic hand-object pose dataset, allowing an evaluation for object occlusion

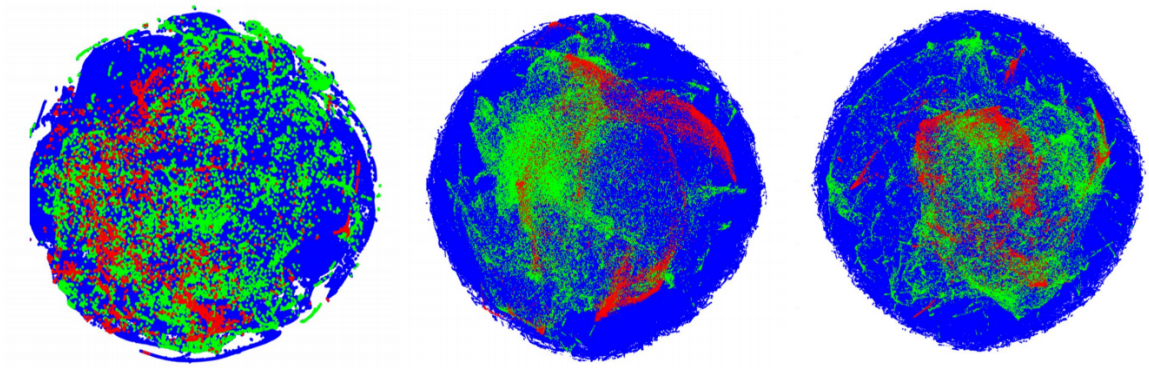


Figure 3.6: t-SNE plot of hand pose dataset coverage demonstrating hand pose space of datasets[16]. Blue: Bighand (Synthetic)[17]; Red: ICVL (Real)[18]; Green: NYU(Real)[19].

to be done on the same dataset. The dataset is annotated with 3D joint locations as well as mesh vertices, so we can also evaluate how well the network learns the hand shape.

(a) No Object Occlusion (b) Some Object Occlusion (c) Full Object Occlusion

Figure 3.7: Samples from the Obman synthetic dataset [12].

In order to test occlusion, we use the Obman dataset to isolate test variables to only the presence of an object. The dataset is rendered both with and without objects in the exact same hand pose as shown in images (a) and (b) in figure 3.7. Comparing object occlusion across two datasets introduced too many variables to conclude the performance of this approach under occlusion.

Visually, to a human, there is enough information about the hand in an occluded image, figure 3.7 (b), to infer the hand pose. We expect the network to still be able to mimic this hand using the MANO model. When it comes to images like image (c), with the hand fully occluded, object analysis is required to infer the grasp.

[12] depends on the presence of an object to infer a hand pose, as they predict object meshes and physically simulate a hand grasp. We evaluate if we can learn the hand pose only based on information about the hand that is visible.

3.4 Experiments

Based on our review of the literature in chapter 2 and design of this MANO based approach in chapter 3, we provide a summary of the hypotheses this research is based on.

1. A deep network will be able to drive the MANO model to mimic hand poses in RGB images with comparable accuracy to other RGB methods. Tested in section 5.2.
2. A deep network will still be able to mimic hand poses under occlusion using the MANO model. Tested in section 5.3.
3. Using the $\vec{\beta}$ shape parameters of the MANO model, the network can learn to fit a subject's hand, improving pose accuracy. Tested in section 5.1.2.
4. Synthetic depth data greatly improves depth-based approaches evaluations. Despite RGB synthetic data not looking very realistic, the network will still be able to generalize and learn how to mimic real hands from this synthetic data. Evaluated in section 5.1.4.

There are three hypotheses that were not tested in this research as a result of time constraints.

1. RGB hand pose estimation can be as accurate as depth-based hand pose estimation. RGB offers more flexibility than depth with higher resolution enabling tracking from greater distances. RGB colour information enables better hand pose estimation under occlusion as a result of easier hand segmentation from the object it is interacting with. Discussed in section 6.1.
2. The network can learn hand pose estimation in both 1st and 3rd person perspectives. The evaluations in this research are only from the 3rd person perspective.
3. This approach can generalize outside of its training data better than other approaches. Discussed in section 6.1.

4 Implementation

We present the implementation details, training processes and benchmark protocols for this research.

All training and benchmarks were run on the same machine throughout the research, table 4.1

CPU	Ryzen 3600, 6c/12t
GPU	GTX 1070ti
RAM	32GB

Table 4.1: Training and Benchmark computer specifications.

The implementation is built with Python 3.7, using PyTorch [56] as the deep learning library to train the network. The ResNet architecture is available in PyTorch and model weights could be loaded from there. The MANO model comes from [2] with code from [3, 57]. We use the evaluation procedures for SOTA comparisons from [20, 58, 59]. We could use code from [59] to load the Obman and STB dataset annotations. We use evaluation protocols from [12] for direct comparisons on the Obman dataset.

4.1 Pipeline

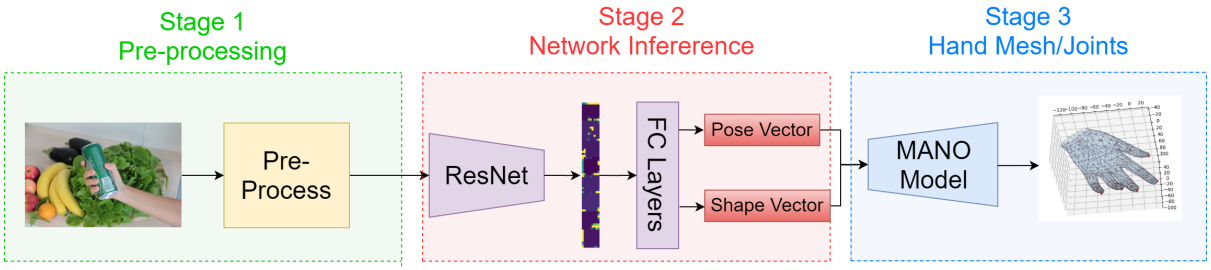


Figure 4.1: Overview of pipeline.

The pipeline consists of three parts, figure 4.1. Stage 1 pre-processes streamed RGB data to the network. We use ResNet-18 for the network architecture, more details in section 4.2. Latent feature vector from the residual network is passed through a fully connected network

to produce the final MANO $\vec{\theta}$ pose and $\vec{\beta}$ shape estimations. These parameters are fed through the MANO layer to produce the resulting hand joints and mesh.

Stage 1 of the network scales and centres the hand in the image. The network in stage 2 is only trained to infer hand pose, it does not learn translations or scales of hands in images. The network learns $\vec{\theta}$ pose parameters and global rotations of a hand. Optionally, the network learns $\vec{\beta}$ shape parameters as well by forwarding the latent feature vectors from ResNet through another fully connected network of 3 layers.

It was out of scope to implement a working solution for stage 1 of the pipeline. For training and benchmarking we used code from [59] to crop and scale hands based on ground truth locations for training. We do present an implementation proposal for stage 1 in section A1.2. We just emulate an implementation in training for the purpose of this research.

The feature maps from ResNet are passed forward into 3 fully connected layers which output the MANO pose $\vec{\theta}$ vector and optionally the shape $\vec{\beta}$ vector. These two vectors are passed into the MANO layer to produce a final mesh prediction.

We use the smallest ResNet architecture, 18 layers, to increase the real-time performance of the implementation. We tested out larger networks in 5.1 and did not see a benefit in using a deeper network. We recorded an average of 0.018 seconds per forward pass on the network on my machine, thus giving us $1/0.018 = 55fps$. This is faster than evaluated run-time speeds of other methods in table 3.1.

4.2 Network implementation

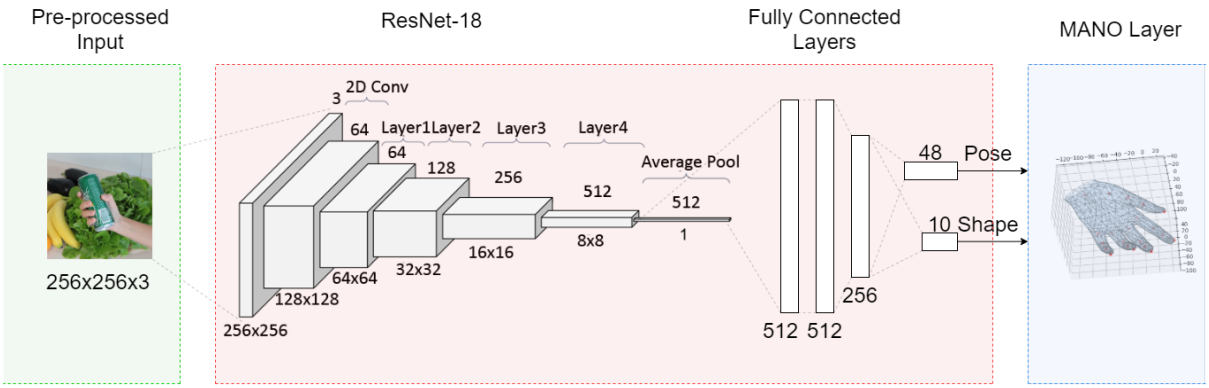


Figure 4.2: Diagram of network architecture.

Input image $x\epsilon\mathbb{R}^{3x256x256}$ is fed into the ResNet-18 network to produce an encoded feature vector. The vector is fed through 3 fully connected layers to output the final MANO parameter vector.

The size of the final output layer depends on the MANO settings. We predict $\vec{rot} = \mathbb{R}^3$ for the global rotation of the hand. $\vec{\theta} = \mathbb{R}^n$ pose vector where $n = \{x\epsilon\mathbb{N}|5 \leq x \geq 45\}$, the

Network Stage	Layer Name	Channels	Vector Size / Stride
Input	Image	3	256x256
ResNet-18	2D Conv + BN + ReLu	64	128x128 / 2
	Max Pool	64	64x64
	Residual Layer 1	64	64x64
	Residual Layer 1	64	64x64 / 2
	Residual Layer 2	128	32x32 / 2
	Residual Layer 2	128	32x32 / 2
	Residual Layer 3	256	16x16 / 2
	Residual Layer 3	256	16x16 / 2
	Residual Layer 4	512	8x8 / 2
	Residual Layer 4	512	8x8 / 2
	Average Pool	512	1
Fully connected layers	Layer 1	512	
	Layer 2	256	
	Layer 3	48	

Table 4.2: Network Architecture. Each residual layer is followed by batch normalization and ReLu activation function.

number of principal components. Optionally $\vec{\beta} = \mathbb{R}^{10}$ shape vector. We use $\vec{\beta} = \vec{\mathbf{0}}^{10}$ vector for the shape parameters if we are not predicting hand shape in the network.

Resulting MANO pose $\vec{\theta}$ and shape $\vec{\beta}$ parameters are fed back into the MANO layer to output the hand mesh and 21 keypoint locations.

4.3 Training

For the scope of this research, the model is trained only to expect left-hand images. Right hands are predicted by flipping the input RGB image along the x-axis to look like a left-hand image to input to the network. The predicted left hand from the network is flipped again along the x-axis to produce a right hand. All training data feature left hands. To predict right hands outside of training, a right-left hand classifier would need to be added to the input pre-processing stage of the network to determine if image flipping is required.

Hands are expected by the network to be approximately scaled and centred. We do not implement a hand segmentation and ROI crop network. Training images are scaled and cropped around the location of the ground truth root palm joint location.

For all training experiments, we used the Adam optimizer, with a learning rate of 10^{-4}. Images are passed into the network with batch size 32. Batch normalization is used between network layers.

We apply data augmentation techniques on input images for training. Firstly, the input image is normalized between 0 and 1 with their respective RGB channels having slightly

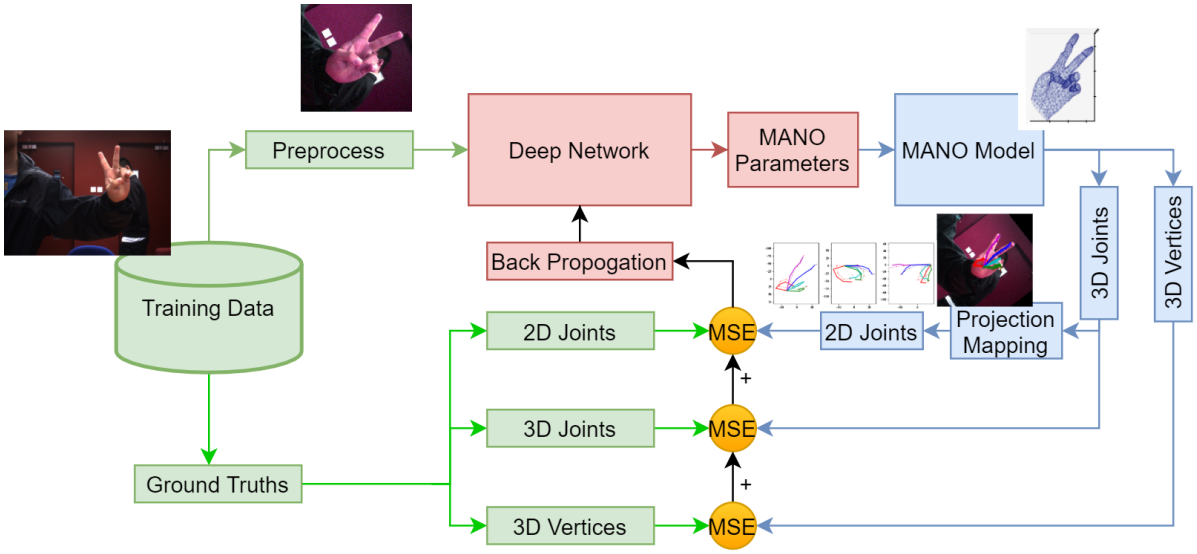


Figure 4.3: Training the model.

varying mean and standard deviations $\vec{mean} = [0.485, 0.456, 0.406]$ and $\vec{std} = [0.229, 0.224, 0.225]$.

- Random Gaussian Blur
- Random offsets on image brightness, saturation, hue and contrast
- Random translation
- Random rotation

When training on the STB dataset, we follow the training protocols from [1, 7, 12, 60]. 15k images are in the training set and 3k images in the test set. The Obman dataset is split into 141k training images and 6k validation images. All results in this research are from evaluating models on the unseen test set to avoid reporting inflated numbers on over-fitted training data. This gives a fair comparison to other approaches trained in the same way.

An element-wise mean squared function $MSE = \frac{1}{n}\sum_{i=1}^{n}(x_i - \hat{x}_i)^2$ is the loss function used. The MANO layer outputs vectors $j \epsilon \mathbb{R}^{21x3}$ of 3d joints and vertices $v \epsilon \mathbb{R}^{778x3}$ from which a loss is calculated against the ground truth annotations.

The STB dataset is annotated with 2D ground truth joint locations $k \epsilon \mathbb{R}^{21x2}$ and Obman made use of projection mapping to generate 2D predictions from the 3D joints. We used their implementation to experiment with. The 2D joint locations did not help the network learn in the end. 3D joints were enough per section 5.1.3.

We have 3 loss functions.

- $\mathcal{L}_j$ MSE 3D joint loss

- $\mathcal{L}_k$ MSE 2D joint loss via 3D $\mapsto$ 2D projection mapping from [12]
- $\mathcal{L}_v$ MSE 3D Vertices loss

The multiple loss functions enables training on any public dataset annotated with either joints or surface meshes. On the STB dataset, we train on 3D joints and 2D joints. On the Obman dataset, we train with 3D joints and Vertices. The main benefit of these multiple loss functions is the ability to train on most public datasets. If a dataset was not annotated with surface mesh vertices, then this approach can just be trained on the 3D joint locations.

The network assumes scale-invariant hands and does not predict global positions of the hand. 3D losses are calculated by centring the hand prediction on the palm key joint. The relative hand coordinates are given by a subtraction of the ground truth annotation $\vec{j^{rel}} = \vec{j^{pred}} - \vec{j^{gt}}$.

4.4 Benchmarking

When running controlled experiments between the two datasets, we use the same evaluation metrics from [1, 12, 20] for direct comparison. When comparing baseline performance between the two models, the training set size is limited to 14976. STB dataset evaluations use 3k images in the test split per [1, 20]. These images are not used in the training set. The Obman evaluations use the same 6k test images Obman uses in their evaluations.

3 metrics are measured during evaluation.

- End-Point-Error (EPE) Mean in mm: $x \epsilon \mathbb{N}$
- End-Point-Error (EPE) Median in mm: $x \epsilon \mathbb{N}$
- Area-Under-Curve (AUC) of Percent-Correct-Keypoints (PCK) curve: $AUC = \{x \epsilon \mathbb{R} | 0 \geq x \leq 1\}$

We did not use a λ weighting for the loss functions.

End-Point-Error measures the euclidean distant error between predicted and ground truth joints in mm. EPE Mean and Median measures the EPE errors for each key joint in each sample in the test set, where t is the number of samples in the test set and j is each joint index.

$$\frac{1}{t}\frac{1}{21}\sum_{i=1}^{t}\sum_{j=1}^{21}\|pred_{ij} - gt_{ij}\| \tag{1}$$

The EPE Median gives some understanding of the distribution of the EPE's. A median lower than the mean indicates a positively skewed distribution of EPE's, implying a higher

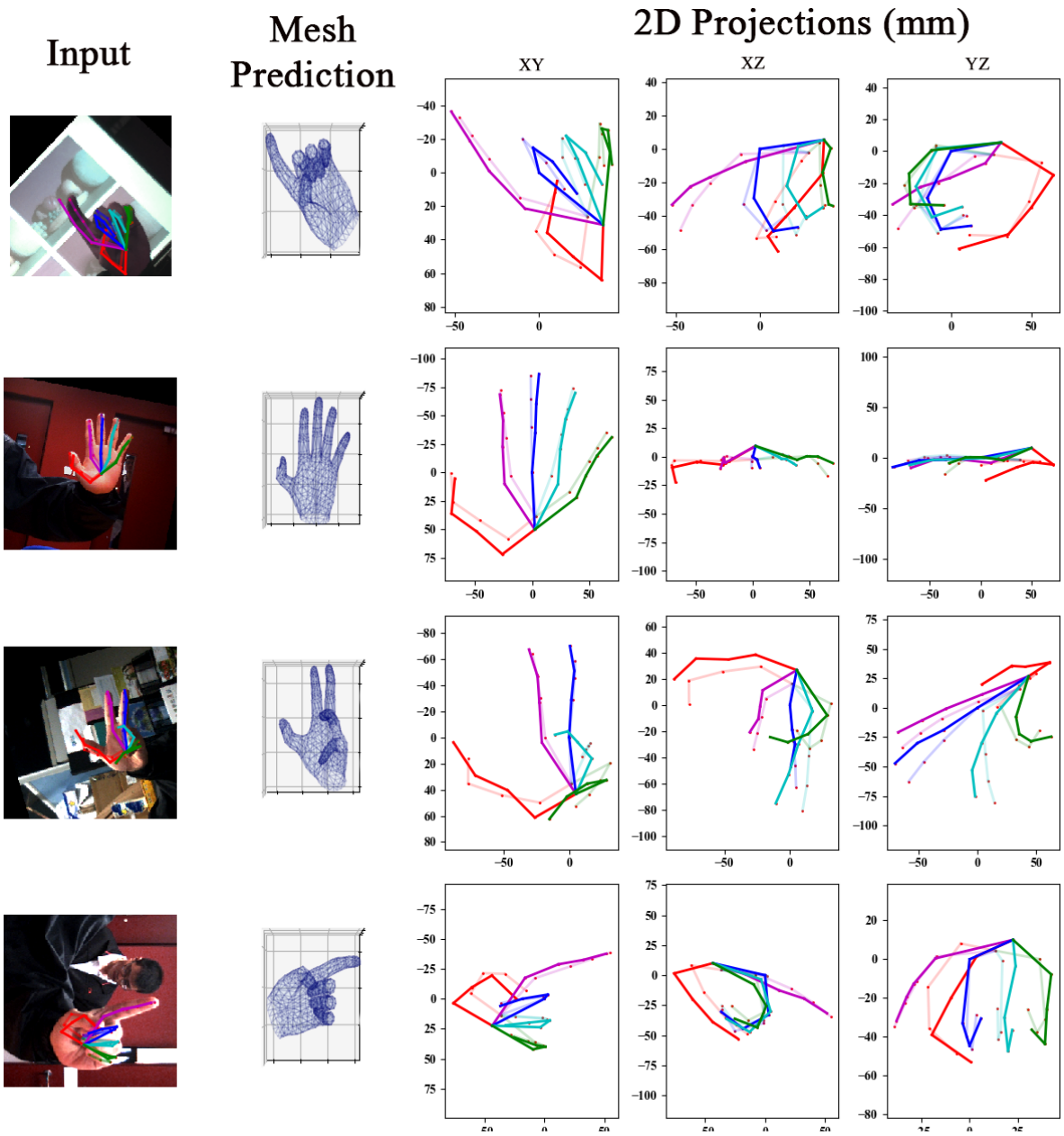


Figure 4.4: Examples of predictions during training on the STB dataset.

percentage of predictions are more accurate, with a fewer number of outliers.

[17, 39] proposes an interesting metric based on the correctness of joints that are visible or hidden to the camera. We could rank the models trained for their ability to interpolate hidden joints. Datasets need joints to be annotated with a hidden or visible attribute though, so we cannot use it in this particular research. The only dataset we've seen with this annotation is the depth-based BigHand2.2 [17]. No RGB datasets that we have examined have this annotation, so we cannot use it for our evaluations.

Area-Under-Curve (AUC) of Percent-Correct-Keypoints (PCK) (abbreviated to AUC from here) is used to determine the best performing models. EPE Mean and Median values fail to properly represent the overall robustness of a model. The percentage of key-points that fall under a particular distance error threshold in mm is recorded and plotted, figure 4.5.

$$PCK = \frac{N_f(\epsilon)}{N} \tag{2}$$

$$AUC = \int_0^{50} \frac{N_f(x)}{N} dx \tag{3}$$

Where $N_f(\epsilon)$ is the number of frames whose joints are within a euclidean distance error threshold ϵ, N = total number of frames.

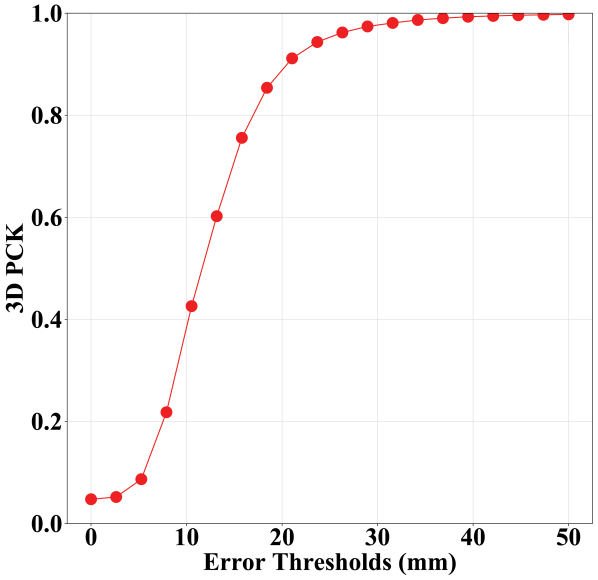


Figure 4.5: Example of PCK curve.

The higher the area under the curve, the larger the ratio of samples succeeding a minimum error threshold. We can use this to determine the reliability of a model under a certain error threshold to not output a poor prediction.

5 Evaluation

We present results from evaluations on public benchmarks discussed in section 4.4.

To control experiments between STB and Obman datasets, dataset size was limited to 14976 samples per epoch for both datasets.

We first report and discuss results on experiments for improving our baseline performance of the network. We then report results from experiments in relation to the configuration of the MANO model. Finally, evaluations against the state of the art methods are presented.

5.1 Establishing baseline

We experimented with various depths of ResNet architectures, 18, 30 and 50 residual layers.

Three models were trained for 20 epochs on the Obman dataset without object occlusion using only 3D joint loss $\mathcal{L}_j$. Models were evaluated on the test split from the Obman dataset.

Architecture	**ResNet-18**	ResNet-34	ResNet-50
AUC	0.536	**0.54**	0.498
Mean Joint Error (mm)	**25.013**	25.21	27.68
Median Error (mm)	21.4193	**21.03**	23.604
Training Time (minutes)	**62**	75	99
Inference Time (seconds)	**0.008**	0.012	0.019

Table 5.1: Table of ResNet architecture performance after 20 epochs.

We use ResNet-18 for subsequent experiments. Deeper networks offered no improvements in evaluation performance, took longer to train and were slower to process frames.

5.1.1 Transfer Learning

[12, 20] benefited by pre-loading their networks with pre-trained weights from the ImageNet challenge. The weights are loaded from PyTorch's published pre-trained models [56]. We

experiment to see what sort of improvements we can expect. We train two models for 30 epochs, with 3d Joint loss $\mathcal{L}_j$ on the Obman dataset.

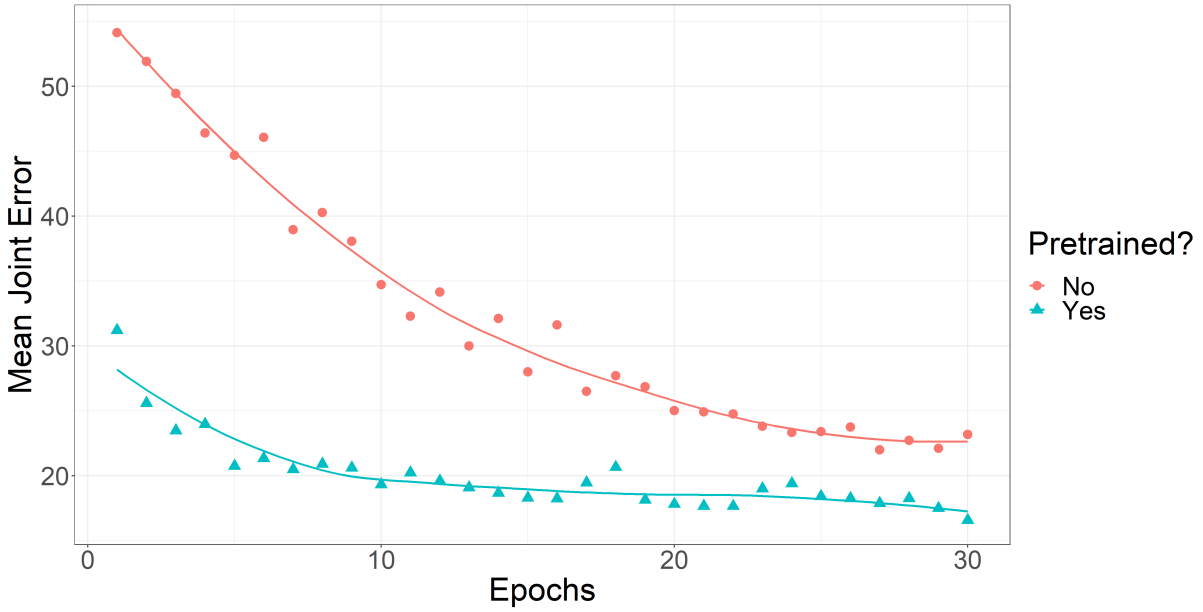


Figure 5.1: Pre-trained ImageNet weights improve performance. Lower is better.

We graph the mean joint error for each model throughout training in figure 5.1. Using pre-trained weights greatly improved network performance. We load pre-trained weights in subsequent experiments. It was quite surprising to see how well transfer learning improved overall accuracy. It does not appear from figure 5.1 that the randomly initialized network would converge to the accuracy of the pre-loaded network in any reasonable amount of time. The pre-loaded weights really help with escaping any local minima in training.

5.1.2 MANO Parameters

Principal Component Analysis

We use PCA on the $\vec{\theta}$ pose parameters of the MANO model. From our hypotheses, we think a lower dimensionality can improve predictions by making learning simpler for the network. We train and evaluate models from 5 up to 45 principal components on both the STB and Obman datasets.

We did not find a lower number of principal components to help, per figure 5.2. The MANO model began to saturate performance at 20 principal components, indicating most of the pose space in the Obman dataset was contained within these principal components. We still generally saw improved accuracy with more principal components. We ended up using 30 principal components on the final STB benchmark as it demonstrated maximal performance.

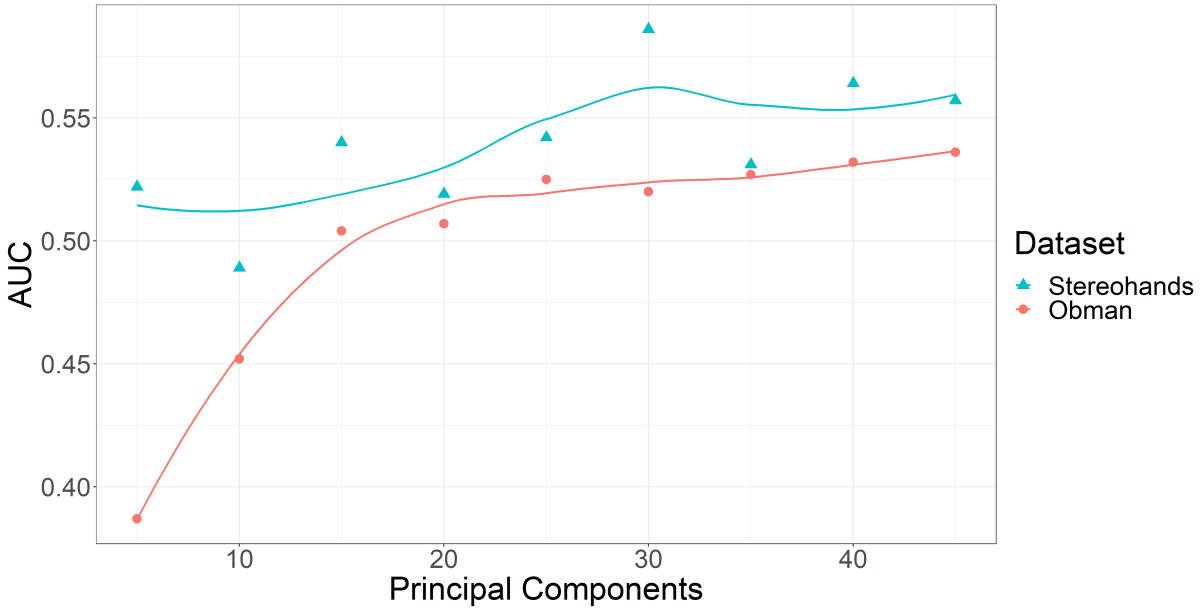


Figure 5.2: AUC at various principal components. Higher is better.

Using a lower number of principal components reduces the possible articulations the MANO hand model can produce. Generally, we saw a higher number of components allows the network to better fit the evaluation data. We speculate that lower than 45 components might result in more stable predictions when used outside of the training data, but this needs a full experiment to confirm.

We did not see the models using a lower number of principal components converge any faster. They all learned at approximately the same rate, so lowering the dimensionality further did not make learning easier or quicker for the network.

Predicting Hand Shape

The MANO model allows us to optionally predict hand shapes with the $\vec{\beta}$ shape parameters. Most RGB datasets are only annotated with joints. We examine how well the network might learn hand shape solely based on 3D joint locations.

We trained both models for 20 epochs on the Obman dataset with just 3D joint loss $\mathcal{L}_j$, no vertex loss $\mathcal{L}_v$.

Train	AUC	EPE Mean
no shape	0.693	15.582
shape	**0.714**	**14.48**

Table 5.2: Shape prediction evaluation results.

We saw a small improvement using shape prediction. This does not confirm that the network can learn to predict hand shapes. We think the system could be biasing itself

towards a single prediction of hand shape that results in an overall improvement in prediction accuracy over the default hand shape. We have no reason to believe the network is currently making correct predictions on hand shape.

We continue to predict $\vec{\beta}$ shape parameters for future experiments to maximize prediction accuracy.

5.1.3 Model Loss

We have various metrics to calculate loss with the MANO model. We were curious if extra 2D joint location and 3D surface mesh annotations are useful and worth the extra effort when producing real datasets. We train each model on the Obman dataset with varying loss functions for 20 epochs.

loss	EPE Mean(mm)
$\mathcal{L}_{2D}$	53.591
$\mathcal{L}_{3D}$	22.1
$\mathcal{L}_{v}$	22.72
$\mathcal{L}_{3D} + \mathcal{L}_{v} + \mathcal{L}_{2D}$	**21.294**

Table 5.3: Evaluating loss functions. Lower is better.

The network could not learn using just 2D joint locations. Either 3D joint or vertex annotations were sufficient for learning.

Using as many loss functions as possible with the annotations a dataset provides marginally improved accuracy, so we use all available loss functions on a given dataset in final evaluations.

5.1.4 Using Synthetic Data

We measure whether synthetic RGB images can supplement training sufficiently. The synthetic RGB hand images in the Obman dataset are noticeably less real looking than real images, example figure 3.7. We want to gain an idea of how these synthetic images might supplement model performance, if at all.

We train three models. The first is trained on the synthetic Obman dataset for 30 epochs. The second is trained on the real STB dataset for 30 epochs. The third model is initially trained for 30 epochs on the synthetic Obman dataset and then for another 30 epochs on the real STB dataset.

All models are evaluated on the STB dataset.

Pre-training on synthetic data showed a significant improvement of $\approx 37.5\%$, per figure 5.4. This large improvement demonstrates the network is capable of taking what it learned from

Data	AUC	EPE Mean(mm)
synth	0.406	34.836
real	0.608	20.48
real+synth	**0.744**	**12.821**

Table 5.4: Effect of pre-training on synthetic data.

mimicking the MANO model to synthetic data and applying that knowledge on real data. This is a good indication that we are not completely over-fitting training data, with some generalization of knowledge across synthetic and real data.

5.2 Stereo Tracking Benchmark

We qualitatively measure the performance of this model on the Stereo Tracking Benchmark following [1, 7, 12] evaluation protocols for a direct comparison.

For the benchmark, we unlock the capped dataset sizes to make use of all data available in the training sets. We still keep the test sets out of the training sets.

We first train the model for 10 epochs on the synthetic Obman, per 5.1.4 dataset with all loss functions $\mathcal{L}_{2d} + \mathcal{L}_j + \mathcal{L}_v + \mathcal{L}_s$, taking advantage of the annotated surface mesh per 5.1.3.

We follow this by training for 90 epochs on the STB dataset, without the vertex loss $\mathcal{L}_v$ as there are no surface mesh annotations.

We used 30 principal components per 5.1.2 and predict shape for both training splits per 5.1.2.

	AUC	EPE Mean(mm)	EPE Median(mm)
HandMesh Supervised[7]	0.998	6.37	-
Obman [12]	0.992	10.0	-
HandMesh Un-supervised [7]	0.974	10.57	-
MANO *	**0.977**	**12.526**	**11.66**
GANerated [20]	0.965	-	-
Hand3d [1]	0.948	-	-

Table 5.5: STB benchmark results.

We display the PCK curve of this MANO approach compared to other methods on the Stereo Tracking Benchmark in figure 5.3. We do not have the curve for HandMesh [7] plotted in figure 5.3. They would otherwise have the highest PCK curve in this plot, per table 5.5.

The HandMesh method uses surface mesh annotations during training to predict perfect hand surface meshes. The STB dataset does not have surface meshes annotated, so the

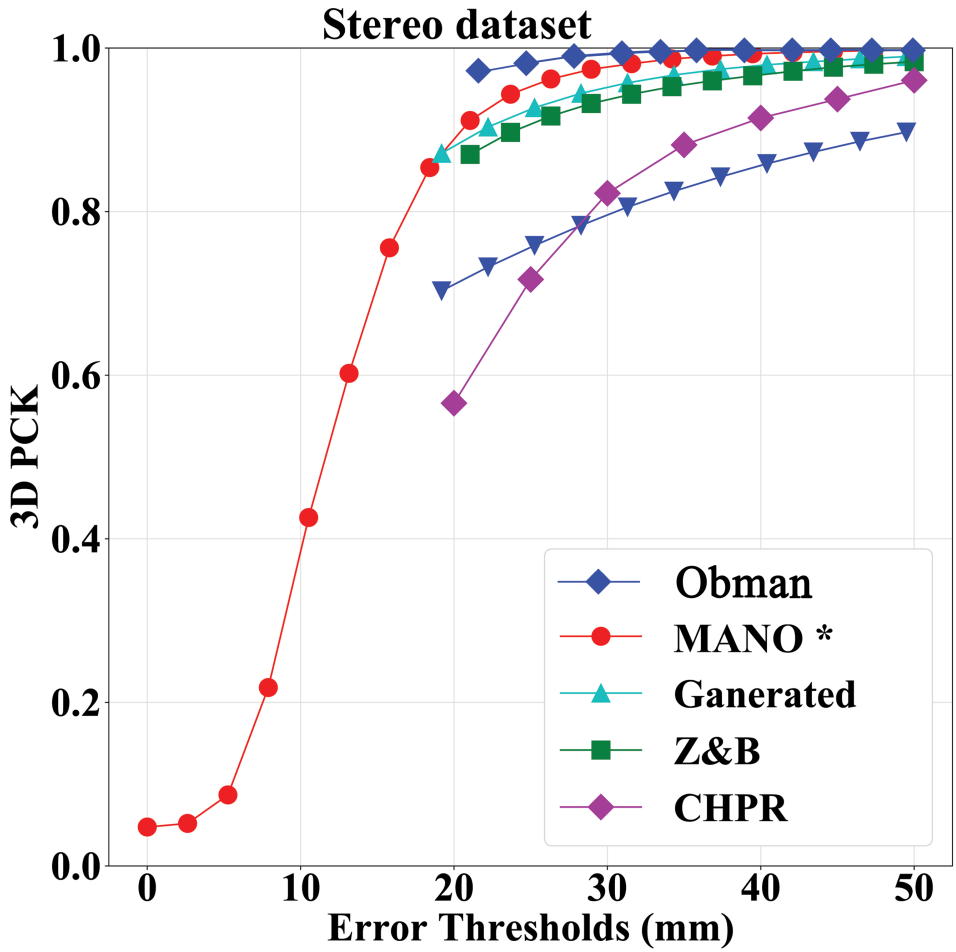


Figure 5.3: Quantitative SOTA benchmark comparison on STB dataset. Higher is better. Evaluations for other SOTA methods from [20]. Methods: Obman[12], GANerated[20], Z&B[1], CHPR[21].

authors generate extra annotations for the surface meshes from the corresponding depth data to aid their approach in training. With this extra supervision in training, they achieved 6.37*mm* mean joint error. Without this extra manual supervision on the STB dataset, they achieved 10.57*mm* of mean joint error. We speculate the improvement from the extra supervision of their approach is a result of their hand mesh over-fitting the subject's hand in the STB dataset and not from more accurate pose predictions. Other than the outlier supervised HandMesh method, this MANO approach benches approximately in the middle of the pack.

5.3 Object Occlusion

To evaluate the ability of the model to track hands interacting with objects, we use the Obman dataset to vary the presence of objects in the evaluation set.

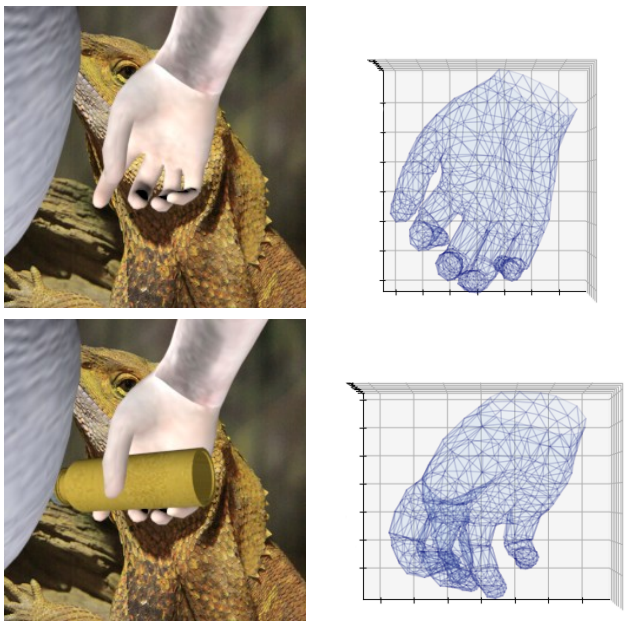
Training on HO images resulted in overall accuracy staying the same compared to H images.

Training data	Evaluation Images	AUC	EPE Mean(mm)
H	H	0.723	14.22
H	HO	0.683	16.69
HO	HO	0.723	14.27

Table 5.6: Model evaluated with and without object occlusion. H=Hand image, HO=Hand-Object image.

The interesting result is the un-occluded hands model only losing $\approx 2.5mm$ of accuracy tested on occluded images. This indicates that the network is still able to predict reasonable hand poses based on visible hand features only, interpolating under occlusion quite well.

Obman evaluates with $11.6mm$ error on the occluded hands evaluation, more accurate than the $14.27mm$ mean joint error of this hand-only approach.



(a) Input Image (b) Predictions

Figure 5.4: Example of prediction without and with object occlusion.

6 Conclusions & Future Work

The goal of this research was to investigate the merits of a deep network controlling the MANO model to mimic hands featured in RGB images for pose estimation.

Evaluations showed this approach was less accurate than Hasson *et al.* [12] and Ge *et al.* [7] on the Stereo Tracking Benchmark, $\approx 12.5mm$ vs $10mm$ and $6.57mm$ mean joint errors respectively. This approach trades accuracy for general versatility and performance. To achieve their $6.57mm$ error, Ge *et al.* had to generate mesh annotations on the STB benchmark. Without these supervised surface meshes, their method achieved $10.57mm$ mean joint error, closer to our $12.5mm$ accuracy, highlighting how much simpler this MANO approach is to train. It is a single network that can be trained with the annotations already available on any public RGB dataset. Their method processes at $\approx 4fps$ on the benchmark machine, slower than the $55fps$ this MANO approach works at.

In section 5.3, the hand model was capable of reasonably strong predictions under object occlusion. With no object occluded hands in the training set, $16.7mm$ mean EPE error was achieved on occluded hands in the Obman dataset. Training with object occluded hands brought the accuracy back to the baseline performance of $14.27mm$ mean EPE on the Obman dataset. This approach was less accurate than Obman's $11.6mm$ mean error with their hand-object mesh estimation. The $16.7mm$ of accuracy is a result of training only on hand images, showing the network can still recognize occluded hand poses based only on visible hand-features. The Obman processed images at $16fps$. slower than the $55fps$ of this single network approach.

We find this MANO based approach can be likened to a jack of all trades. A deep network can produce reliable hand pose estimations from RGB images by mimicking hands with the MANO hand-model. This approach was robust against partially obscured hands. The model is simpler to train, easier to implement with a single network and is more run-time performant compared to other RGB methods. This approach trades these advantages for an overall lower accuracy in benchmarks against the state of the art methods in their respective domains.

6.1 Future

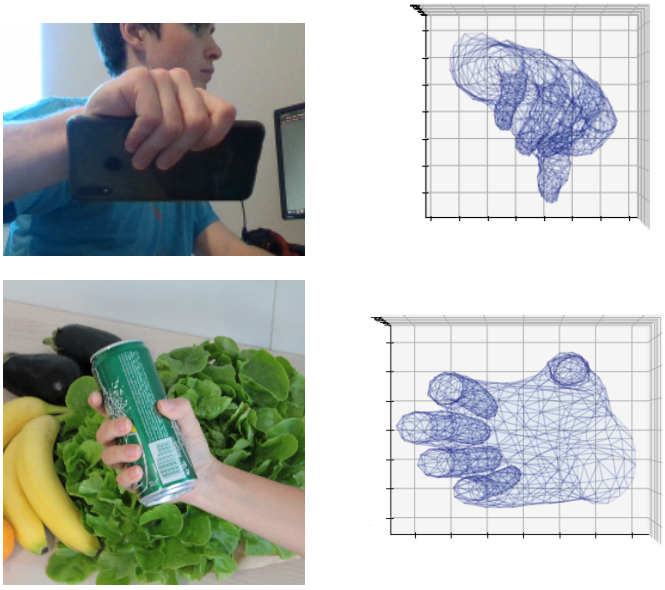
The results in chapter 5 have been promising, validating future research.

We did not manage to evaluate this approach in the 1st person perspective. Hands in the 1st person can be heavily self occluded. We would suggest further evaluations on the 1st person GANerated [20] synthetic dataset and real 1st person FHAD [13] hand-object interaction dataset.

We still hypothesize RGB to be a more promising modality for hand pose estimation in the future over depth. We think RGB data contains more information about the subtle articulations in hand poses, particularly under object occlusion. Most RGB datasets also have corresponding depth data, making a direct comparison possible on the same dataset. A comparative study on the potential that RGB images provide over depth-images for would be insightful.

We would like to see the network trained on many more datasets. It would be interesting to see how robust this approach could be. In appendix A1.1 we present a training procedure on various datasets that could potentially produce reliable hand tracking.

Out of curiosity, we processed the demo images we used when examining the other state of the art methods in figure 3.2 on this MANO based approach. Figure 6.1 indicates the system is capable of some generalization outside of its prior knowledge.



(a) Input Images (b) Predictions

Figure 6.1: Example outputs from the system on unseen data

The surface mesh approach from Ge *et al.* is a lot more accurate on the STB dataset than the MANO approach. When it was tested on an occluded hand in figure 3.2, it was unable

to produce any kind of hand. Figure 6.1 indicates that this MANO model-based approach could be more resistant to failure. An evaluation on the surface mesh approach Ge *et al.* proposes on occluded hands should be done to evaluate its reliability. An experiment to compare the robustness of this approach outside its training data relative to the state of the art needs to be setup. We hypothesized this approach could generalize better than other methods but were unable to confirm or deny this hypothesis in this thesis.

6.2 Limitations and discussions

As mentioned, the pre-processing stage of this approach was mocked for training and evaluating. See Stage 1 of figure 4.1. We discuss a potential implementation for this pre-processing stage in appendix A1.2. Correct input to the network is assumed for the evaluations in chapter 5. Errors in the pre-processing stage of the network would propagate and affect the overall accuracy of the implementation.

Any hand pose estimation implementation needs to be able to generalize outside of its training data. We found to some degree the model generalized hand pose outside its training set, evident from the model transferring knowledge between synthetic and real data in section 5.1.4. The network still recognized hand poses under object occlusions despite not being trained on object occluded hands, per section 5.3. Examples in figure 6.1 are promising, but we cannot conclude the ability for this approach to generalize compared to the state of the art. We did not have the time to set up and run experiments on generalization compared to state of the art.

Public benchmarks like STB reward methods that are more capable of over-fitting, rather than evaluating objectively an approach to general hand pose estimation. We do not see the state of the art methods being evaluated outside of ideal benchmark conditions, and so we cannot make a comparison on generalization. Accuracy in hand pose estimation research is saturating on benchmarks around $9mm$ and $10mm$ mean joint error. For more reliability in hand pose estimation, we would like to see more research on how well these approaches generalize outside their training data.

Bibliography

[1] Christian Zimmermann and Thomas Brox. Learning to estimate 3d hand pose from single rgb images. In *IEEE International Conference on Computer Vision (ICCV)*, 2017. URL `https://lmb.informatik.uni-freiburg.de/projects/hand3d/`. https://arxiv.org/abs/1705.01389.

[2] Javier Romero, Dimitrios Tzionas, and Michael J. Black. Embodied hands: Modeling and capturing hands and bodies together. *ACM Transactions on Graphics, (Proc. SIGGRAPH Asia)*, 36(6), November 2017.

[3] Hands 2019 challenge, 2019. URL `https://sites.google.com/view/hands2019/challenge`. Updated . Accessed October 2019.

[4] Iason Oikonomidis, Nikolaos Kyriazis, and Antonis Argyros. Efficient model-based 3d tracking of hand articulations using kinect. In *BMVC*, volume 1, 01 2011. doi: 10.5244/C.25.101.

[5] Srinath Sridhar, Antti Oulasvirta, and Christian Theobalt. Interactive markerless articulated hand motion tracking using rgb and depth data. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, December 2013. URL `http://handtracker.mpi-inf.mpg.de/projects/handtracker_iccv2013/`.

[6] Jonathan Taylor, Lucas Bordeaux, Thomas Cashman, Bob Corish, Cem Keskin, Eduardo Soto, David Sweeney, Julien Valentin, Benjamin Luff, Arran Topalian, Erroll Wood, Sameh Khamis, Pushmeet Kohli, Toby Sharp, Shahram Izadi, Richard Banks, Andrew Fitzgibbon, and Jamie Shotton. Efficient and precise interactive hand tracking through joint, continuous optimization of pose and correspondences. *ACM Transactions on Graphics (TOG) - Proceedings of ACM SIGGRAPH 2016*, 35, July 2016. URL `https://www.microsoft.com/en-us/research/publication/efficient-precise-interactive-hand-tracking-joint-continuous-optimization-pose-co`

[7] Liuhao Ge, Zhou Ren, Yuncheng Li, Zehao Xue, Yingying Wang, Jianfei Cai, and

Junsong Yuan. 3d hand shape and pose estimation from a single RGB image. *CoRR*, abs/1903.00812, 2019. URL http://arxiv.org/abs/1903.00812.

[8] Toby Sharp, Yichen Wei, Daniel Freedman, Pushmeet Kohli, Eyal Krupka, Andrew Fitzgibbon, Shahram Izadi, Cem Keskin, Duncan Robertson, Jonathan Taylor, Jamie Shotton, David Kim, Christoph Rhemann, Ido Leichter, and Alon Vinnikov. Accurate, robust, and flexible real-time hand tracking. In *CHI '15*, 04 2015. doi: 10.1145/2702123.2702179.

[9] Gyeongsik Moon, Ju Yong Chang, and Kyoung Mu Lee. V2v-posenet: Voxel-to-voxel prediction network for accurate 3d hand and human pose estimation from a single depth map. *CoRR*, abs/1711.07399, 2017. URL http://arxiv.org/abs/1711.07399.

[10] Zhaohui Zhang, Shipeng Xie, Mingxiu Chen, and Haichao Zhu. Handaugment: A simple data augmentation method for depth-based 3d hand pose estimation, 2020.

[11] Tomas Simon, Hanbyul Joo, Iain A. Matthews, and Yaser Sheikh. Hand keypoint detection in single images using multiview bootstrapping. *CoRR*, abs/1704.07809, 2017. URL http://arxiv.org/abs/1704.07809.

[12] Yana Hasson, Gül Varol, Dimitris Tzionas, Igor Kalevatykh, Michael J. Black, Ivan Laptev, and Cordelia Schmid. Learning joint reconstruction of hands and manipulated objects. In *CVPR*, 2019.

[13] Guillermo Garcia-Hernando, Shanxin Yuan, Seungryul Baek, and Tae-Kyun Kim. First-person hand action benchmark with RGB-D videos and 3d hand pose annotations. *CoRR*, abs/1704.02463, 2017. URL http://arxiv.org/abs/1704.02463.

[14] Awesome hand pose github repository, 2020. URL https://github.com/xinghaochen/awesome-hand-pose-estimation. Accessed January 2020.

[15] J. Zhang, J. Jiao, M. Chen, L. Qu, X. Xu, and Q. Yang. A hand pose tracking benchmark from stereo matching. In *2017 IEEE International Conference on Image Processing (ICIP)*, pages 982–986, Sep. 2017. doi: 10.1109/ICIP.2017.8296428.

[16] Qi Ye. *3D hand pose estimation using convolutional neural networks*. PhD thesis, Imperial College London, 2018.

[17] Shanxin Yuan, Qi Ye, Björn Stenger, Siddhant Jain, and Tae-Kyun Kim. Bighand2.2m benchmark: Hand pose dataset and state of the art analysis. *CoRR*, abs/1704.02612, 2017. URL http://arxiv.org/abs/1704.02612.

[18] D. Tang, H. J. Chang, A. Tejani, and T. Kim. Latent regression forest: Structured estimation of 3d articulated hand posture. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 3786–3793, June 2014. doi: 10.1109/CVPR.2014.490.

[19] Jonathan Tompson, Murphy Stein, Yann Lecun, and Ken Perlin. Real-time continuous pose recovery of human hands using convolutional networks. *ACM Transactions on Graphics*, 33, August 2014.

[20] Franziska Mueller, Florian Bernard, Oleksandr Sotnychenko, Dushyant Mehta, Srinath Sridhar, Dan Casas, and Christian Theobalt. Ganerated hands for real-time 3d hand tracking from monocular rgb. In *Proceedings of Computer Vision and Pattern Recognition (CVPR)*, June 2018. URL `https://handtracker.mpi-inf.mpg.de/projects/GANeratedHands/`.

[21] X. Sun, Y. Wei, Shuang Liang, X. Tang, and J. Sun. Cascaded hand pose regression. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 824–832, June 2015. doi: 10.1109/CVPR.2015.7298683.

[22] Xingyi Zhou, Qingfu Wan, Wei Zhang, Xiangyang Xue, and Yichen Wei. Model-based deep hand pose estimation. *CoRR*, abs/1606.06854, 2016. URL `http://arxiv.org/abs/1606.06854`.

[23] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. Imagenet large scale visual recognition challenge, 2014.

[24] C. Choi, S. H. Yoon, C. Chen, and K. Ramani. Robust hand pose estimation during the interaction with an unknown object. In *2017 IEEE International Conference on Computer Vision (ICCV)*, pages 3142–3151, Oct 2017. doi: 10.1109/ICCV.2017.339.

[25] Marco Santello, Martha Flanders, and John F. Soechting. Postural hand synergies for tool use. *Journal of Neuroscience*, 18(23):10105–10115, 1998. ISSN 0270-6474. doi: 10.1523/JNEUROSCI.18-23-10105.1998. URL `https://www.jneurosci.org/content/18/23/10105`.

[26] M. Schröder, J. Maycock, H. Ritter, and M. Botsch. Real-time hand tracking using synergistic inverse kinematics. In *2014 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5447–5454, May 2014. doi: 10.1109/ICRA.2014.6907660.

[27] Matthew Loper, Naureen Mahmood, Javier Romero, Gerard Pons-Moll, and Michael J. Black. SMPL: A skinned multi-person linear model. *ACM Trans. Graphics (Proc. SIGGRAPH Asia)*, 34(6):248:1–248:16, October 2015.

[28] Shanxin Yuan, Guillermo Garcia-Hernando, Björn Stenger, Gyeongsik Moon, Ju Yong Chang, Kyoung Mu Lee, Pavlo Molchanov, Jan Kautz, Sina Honari, Liuhao Ge, Junsong Yuan, Xinghao Chen, Guijin Wang, Fan Yang, Kai Akiyama, Yang Wu, Qingfu Wan, Meysam Madadi, Sergio Escalera, Shile Li, Dongheui Lee, Iason Oikonomidis, Antonis A. Argyros, and Tae-Kyun Kim. Depth-based 3d hand pose estimation: From current achievements to future goals. *CoRR*, abs/1712.03917, 2017. URL `http://arxiv.org/abs/1712.03917`.

[29] James Supancic, Grégory Rogez, Yi Yang, Jamie Shotton, and Deva Ramanan. Depth-based hand pose estimation: Methods, data, and challenges. *International Journal of Computer Vision*, 126:1180–, 09 2018. doi: 10.1007/s11263-018-1081-7.

[30] L. Ge, H. Liang, J. Yuan, and D. Thalmann. 3d convolutional neural networks for efficient and robust hand pose estimation from single depth images. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5679–5688, July 2017. doi: 10.1109/CVPR.2017.602.

[31] L. Ge, Y. Cai, J. Weng, and J. Yuan. Hand pointnet: 3d hand pose estimation using point sets. In *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8417–8426, June 2018. doi: 10.1109/CVPR.2018.00878.

[32] D. Tang, T. Yu, and T. Kim. Real-time articulated hand pose estimation using semi-supervised transductive regression forests. In *2013 IEEE International Conference on Computer Vision*, pages 3224–3231, Dec 2013. doi: 10.1109/ICCV.2013.400.

[33] Srinath Sridhar, Franziska Mueller, Michael Zollhoefer, Dan Casas, Antti Oulasvirta, and Christian Theobalt. Real-time joint tracking of a hand manipulating an object from rgb-d input. In *Proceedings of European Conference on Computer Vision (ECCV)*, October 2016. URL `http://handtracker.mpi-inf.mpg.de/projects/RealtimeHO/`.

[34] C. Choi, A. Sinha, J. H. Choi, S. Jang, and K. Ramani. A collaborative filtering approach to real-time hand pose estimation. In *2015 IEEE International Conference on Computer Vision (ICCV)*, pages 2336–2344, Dec 2015. doi: 10.1109/ICCV.2015.269.

[35] Chin-Seng Chua, Haiying Guan, and Yeong-Khing Ho. Model-based 3d hand posture estimation from a single 2d image. *Image and Vision Computing*, 20:191–202, 03 2002. doi: 10.1016/S0262-8856(01)00094-4.

[36] Jonathan Tompson, Murphy Stein, Yann Lecun, and Ken Perlin. Real-time continuous pose recovery of human hands using convolutional networks. *ACM Trans. Graph.*, 33 (5), September 2014. ISSN 0730-0301. doi: 10.1145/2629500. URL `https://doi.org/10.1145/2629500`.

[37] Julien Valentin, Angela Dai, Matthias Nießner, Pushmeet Kohli, Philip Torr, Shahram Izadi, and Cem Keskin. Learning to navigate the energy landscape. *arXiv preprint arXiv:1603.05772*, 2016.

[38] Bardia Doosti. Hand pose estimation: A survey. *CoRR*, abs/1903.01013, 2019. URL `http://arxiv.org/abs/1903.01013`.

[39] Shanxin Yuan. *3D hand pose estimation: methods, datasets, and challenges*. PhD thesis, Imperial College London, 2018.

[40] Shanxin Yuan, Björn Stenger, and Tae-Kyun Kim. Rgb-based 3d hand pose estimation via privileged learning with depth images. *CoRR*, abs/1811.07376, 2018. URL `http://arxiv.org/abs/1811.07376`.

[41] Franziska Mueller, Dushyant Mehta, Oleksandr Sotnychenko, Srinath Sridhar, Dan Casas, and Christian Theobalt. Real-time hand tracking under occlusion from an egocentric rgb-d sensor. In *Proceedings of International Conference on Computer Vision (ICCV)*, October 2017. URL `http://handtracker.mpi-inf.mpg.de/projects/OccludedHands/`.

[42] Shreyas Hampali, Markus Oberweger, Mahdi Rad, and Vincent Lepetit. Ho-3d: A multi-user, multi-object dataset for joint 3d hand-object pose estimation. *arXiv preprint arXiv:1907.01481*, 2019.

[43] Christian Zimmermann, Duygu Ceylan, Jimei Yang, Bryan Russell, Max Argus, and Thomas Brox. Freihand: A dataset for markerless capture of hand pose and shape from single rgb images. In *The IEEE International Conference on Computer Vision (ICCV)*, October 2019.

[44] Franziska Mueller, Dushyant Mehta, Oleksandr Sotnychenko, Srinath Sridhar, Dan Casas, and Christian Theobalt. Real-time hand tracking under occlusion from an egocentric rgb-d sensor. In *Proceedings of International Conference on Computer Vision (ICCV)*, October 2017. URL `http://handtracker.mpi-inf.mpg.de/projects/OccludedHands/`.

[45] Jamie Shotton, Andrew Fitzgibbon, Mat Cook, Toby Sharp, Mark Finocchio, Richard Moore, Alex Kipman, and Andrew Blake. Real-time human pose recognition in parts from single depth images. In *CVPR 2011*, volume 56, pages 1297–1304, 06 2011. doi: 10.1109/CVPR.2011.5995316.

[46] James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95-International Conference on Neural Networks*, volume 4, pages 1942–1948. IEEE, 1995.

[47] C. Qian, X. Sun, Y. Wei, X. Tang, and J. Sun. Realtime and robust hand tracking from depth. In *2014 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1106–1113, 2014.

[48] Imagenet large scale visual recognition challenge (ilsvrc), 2020. URL `http://www.image-net.org/challenges/LSVRC/`. Accessed January 2020.

[49] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 1097–1105. Curran Associates, Inc., 2012. URL `http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf`.

[50] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition, 2014.

[51] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Computer Vision and Pattern Recognition (CVPR)*, 2015. URL `http://arxiv.org/abs/1409.4842`.

[52] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition, 2015.

[53] E. Todorov and Z. Ghahramani. Analysis of the synergies underlying complex hand manipulation. In *The 26th Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, volume 2, pages 4637–4640, Sep. 2004. doi: 10.1109/IEMBS.2004.1404285.

[54] M. de La Gorce, D. J. Fleet, and N. Paragios. Model-based 3d hand pose estimation from monocular video. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 33(9):1793–1805, 2011.

[55] Linlin Yang, Shile Li, Dongheui Lee, and Angela Yao. Aligning latent spaces for 3d hand pose estimation. In *The IEEE International Conference on Computer Vision (ICCV)*, October 2019.

[56] Pytorch website, 2020. URL `https://pytorch.org/`. Accessed January 2020.

[57] manopth github repository, 2019. URL `https://github.com/hassony2/manopth`. Accessed January 2020.

[58] hand3d github repository, 2017. URL https://github.com/lmb-freiburg/hand3d. Accessed January 2020.

[59] Obman train github repository, 2020. URL https://github.com/hassony2/obman_train. Accessed January 2020.

[60] Umar Iqbal, Pavlo Molchanov, Thomas Breuel, Juergen Gall, and Jan Kautz. Hand pose estimation via latent 2.5d heatmap regression, 2018.

A1 Appendix

A1.1 Training

We expect reliable hand pose estimation results from training on a wider variety of datasets. We propose the following datasets to provide wide coverage of articulations under first and third-person POV's as well as under occlusion.

We would start by training on synthetic datasets to initially learn a wider variety of un-occluded articulations. GANerated [20] is a first-person synthetic hand dataset generated from a Generate-Adversarial-Network (GAN). The Rendered-Handpose-Dataset (RHD) [1] is a synthetic third-person perspective dataset.

To learn hand shapes, we would train on the Obman [12] synthetic dataset without object occlusions. FreiHAND [43] is another mesh annotated dataset that is markerless and real. The mesh annotations should guide stronger shape predictions.

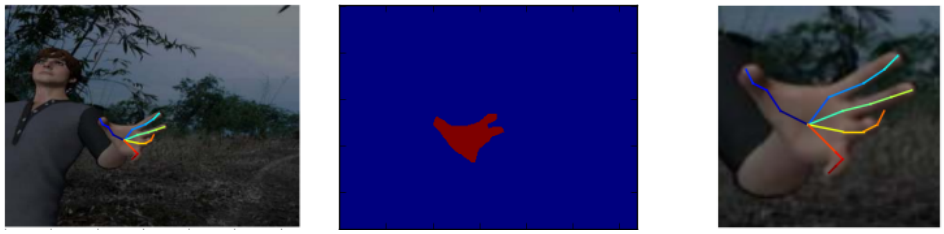
We would also train on the following object occluded datasets. HO-3D [42] is a markerless hand-object real hand dataset from the 3rd person perspective. FHAD [13] is a first-person markered hand-object real hand dataset. The FHAD dataset, as shown in section 2.3 figure 2.15, contains hands covered in magnetic sensors. We would not train on this dataset to avoid the model learning joint positions based on magnetic sensors present on the hand. We would, however, evaluate the model on this dataset. It would provide insight on how well the model generalizes a hand to unseen data under hands occluded in a manner not present in training data.

Based on our benchmarking, the system can learn quite quickly on a smaller amount of data. It only took the network 3 hours, 12 minutes on my machine to achieve the benchmark performance numbers from 5.2. On dedicated professional-grade hardware, training times would be magnitudes faster. We would estimate just a few hundred hours on all the above datasets for a production-ready network.

A1.2 Stage 1 Pre-Processing implementation

The pose network in stage 2 of figure 4.1 only predicts hand global rotation $\vec{r}$, pose $\vec{\theta}$, and shape $\vec{\beta}$ vectors. No predictions are made for the scale and/or translation of hands. The network is trained on hands that are approximately scaled and translated to the centre of the image.

To run outside of a benchmark, an automatic hand cropping module would need to be implemented to process an input stream of RGB inputs. In the wild pose estimation can still be accomplished at the moment with the current system if the subject keeps their hand roughly centred in the image.



(a) Input (b) Hand Segmentation (c) Center of mass crop

Figure A1.1: Example of hand cropping from [1].

[1] makes use of a hand segmentation network with great success, and subsequent cropping based on the centre of mass on the binary mask, figure A1.1.

Some further work should be done to evaluate the performance of the system with a similar hand segmentation/cropping module installed. Use of a segmentation network like this could produce multiple hand crops for different hands in the images allowing for multiple hand pose estimation in the same image. Left/right-hand classification would also have to be done to flip the input to left-hand images for the network. Small errors in the pre-processing network would propagate throughout the rest of the system. Research in this thesis assumed correct input. We would have to observe the downstream effects of errors in the pre-processing stage for accuracy outside of benchmarks.